Machine Intelligence 1

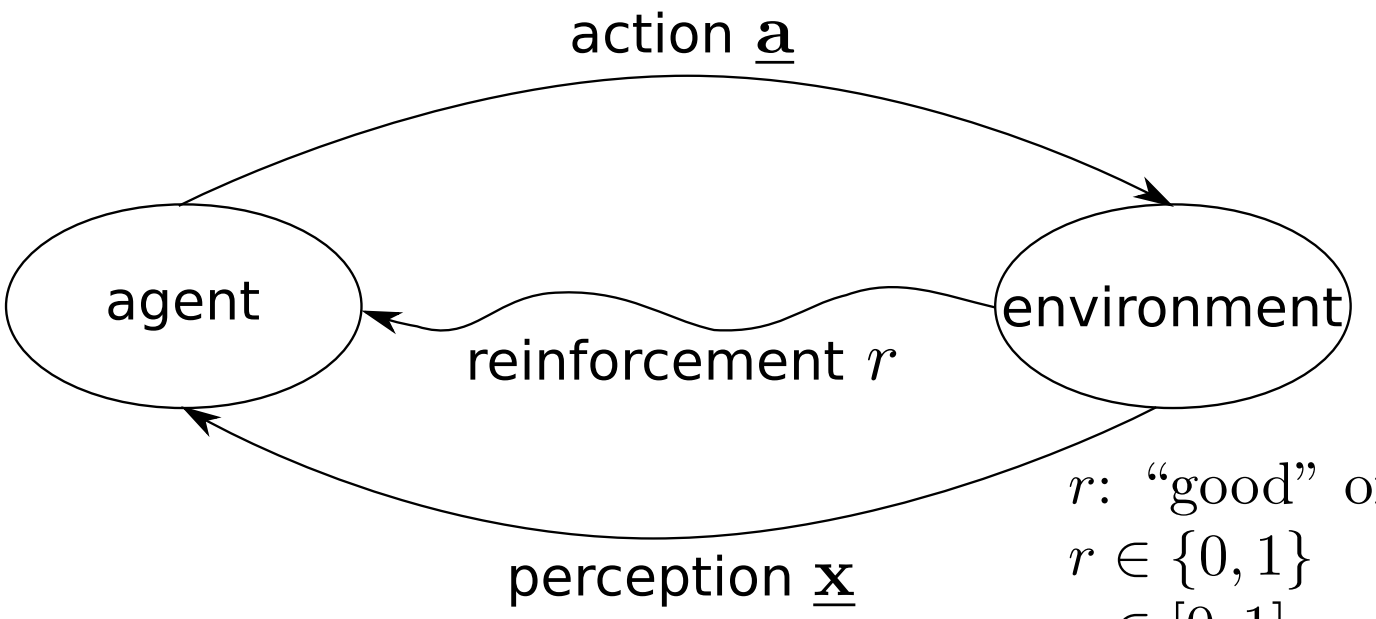
4.1 Reinforcement Learning – Valuation

Prof. Dr. Klaus Obermayer

Fachgebiet Neuronale Informationsverarbeitung (NI)

WS 25/26

Reinforcement learning



r: "good" or "bad"

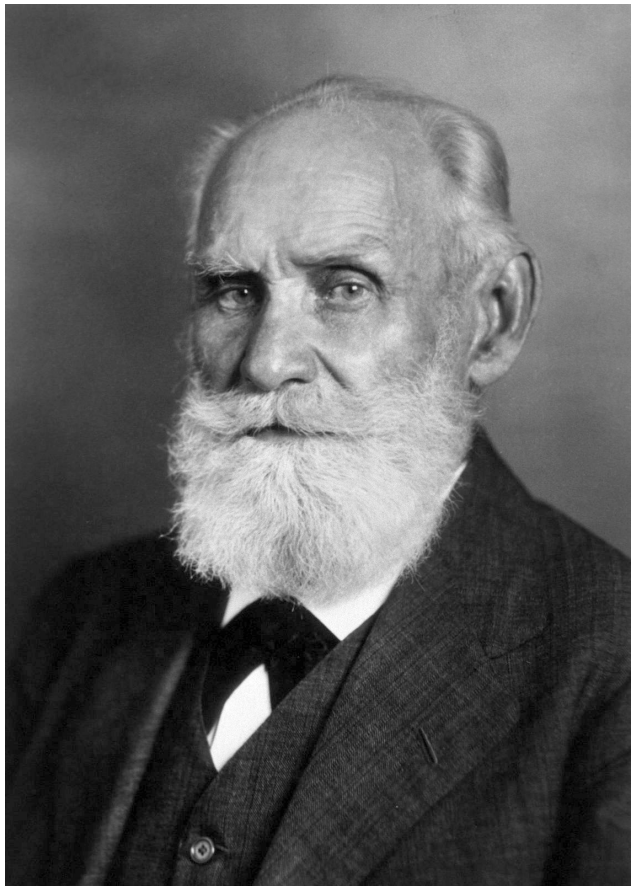
$r \in \{0, 1\}$

$r \in [0, 1]$

$r \in \mathbb{R}$

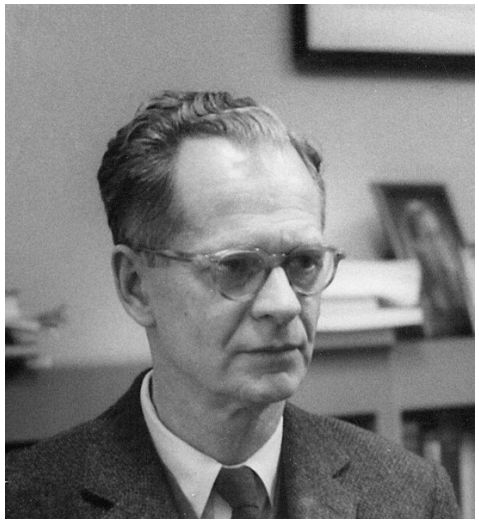
Classical conditioning

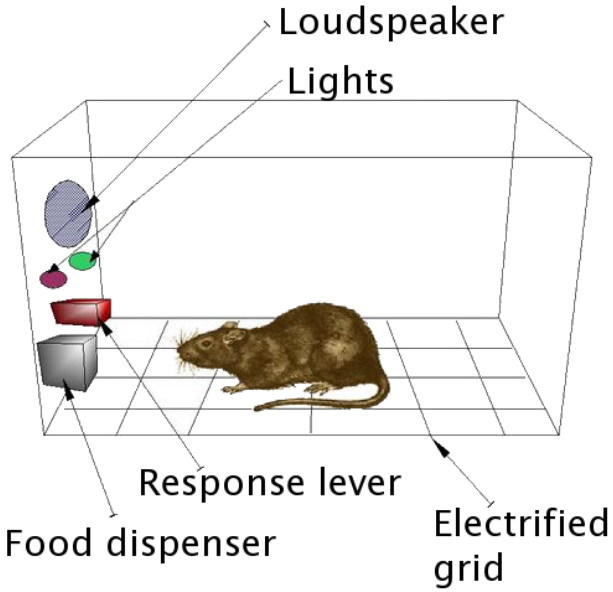
- Ivan Pavlov (1849–1936)
- conditioned stimulus (neutral)
- unconditioned stimulus (rewarding)
- experience reinforces *involuntary response*
- animals learn to *expect* reward



Operant conditioning

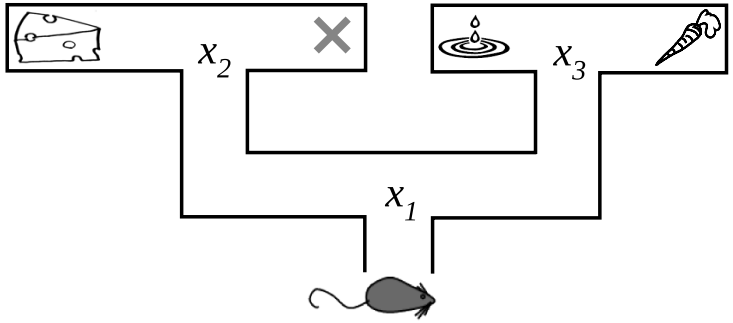
- Burrhus Frederic Skinner (1904–1990)
- animal acts voluntarily
- actions are rewarded or punished
- experience reinforces *voluntary behavior*
- animals learn how to *achieve* reward





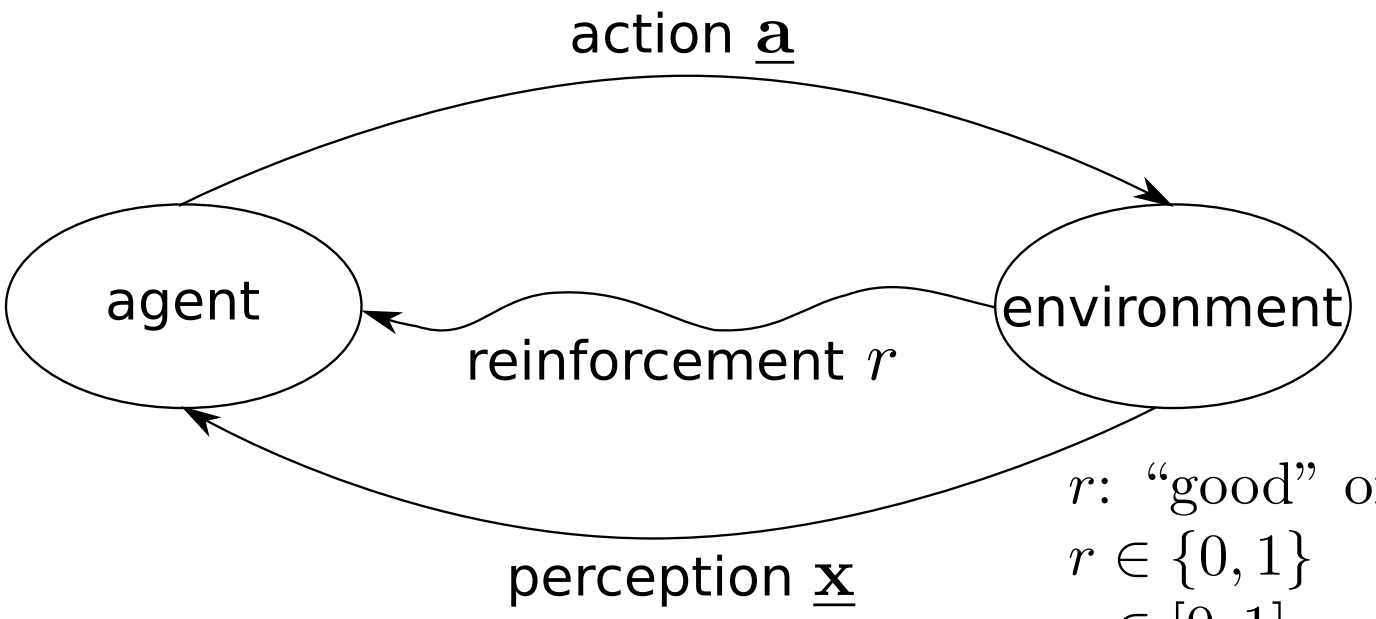
Future rewards

- not all decisions are immediately rewarded
  - decision in **state** x_1 is crucial, but not rewarded
- some decisions require foresight
  - future reward of decision in x_1 depends on decisions in x_2 and x_3
- animal must *delay* the reinforcement of behavior



(see Dayan and Niv, 2008; Dayan, 2008)

Reinforcement learning



r: "good" or "bad"

$r \in \{0, 1\}$

$r \in [0, 1]$

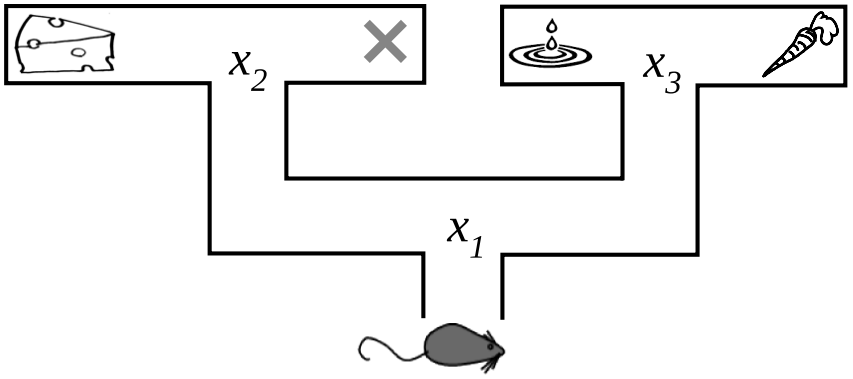
$r \in \mathbb{R}$

4.1.1 Markov Decision Processes

Markov decision processes

A Markov decision process (MDP) consists of

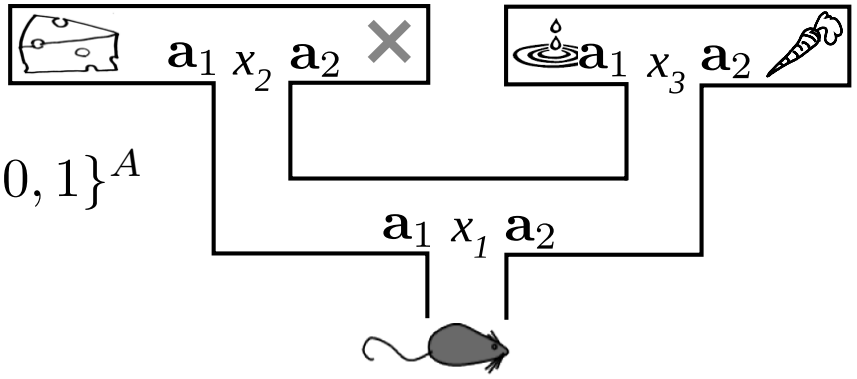
- a set of **states** $\underline{\mathbf{x}} \in \mathcal{X}$,
  - e.g. $\mathcal{X} := \{\underline{\mathbf{x}}_1, \ldots, \underline{\mathbf{x}}_S\} \subset \{0,1\}^S$ with 1-out-of-S encoding



Markov decision processes

A Markov decision process (MDP) consists of

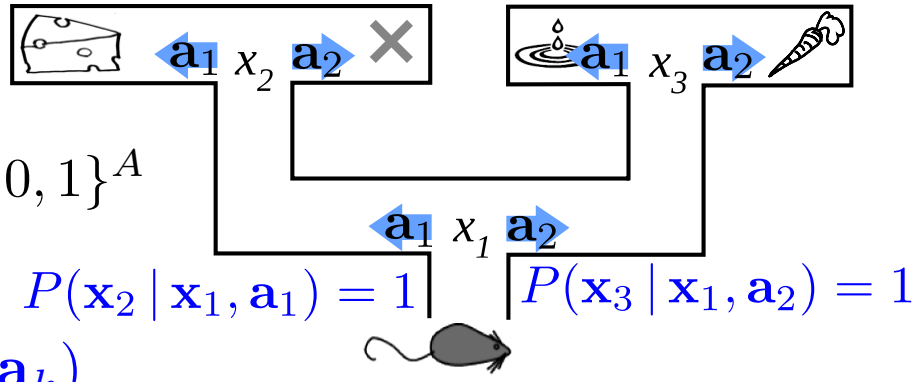
- a set of **states** $\underline{\mathbf{x}} \in \mathcal{X}$,
  - e.g. $\mathcal{X} := \{\underline{\mathbf{x}}_1, \ldots, \underline{\mathbf{x}}_S\} \subset \{0,1\}^S$ with 1-out-of-S encoding
- a set of **actions** $\underline{\mathbf{a}} \in \mathcal{A}$,
  - e.g. $\mathcal{A} := \{\underline{\mathbf{a}}_1, \ldots, \underline{\mathbf{a}}_A\} \subset \{0,1\}^A$ with 1-out-of-A encoding



Markov decision processes

A Markov decision process (MDP) consists of

- a set of **states** $\underline{\mathbf{x}} \in \mathcal{X}$,
  - e.g. $\mathcal{X} := \{\underline{\mathbf{x}}_1, \ldots, \underline{\mathbf{x}}_S\} \subset \{0,1\}^S$ with 1-out-of-S encoding
- a set of **actions** $\underline{\mathbf{a}} \in \mathcal{A}$,
  - e.g. $\mathcal{A} := \{\underline{\mathbf{a}}_1, \ldots, \underline{\mathbf{a}}_A\} \subset \{0,1\}^A$ with 1-out-of-A encoding

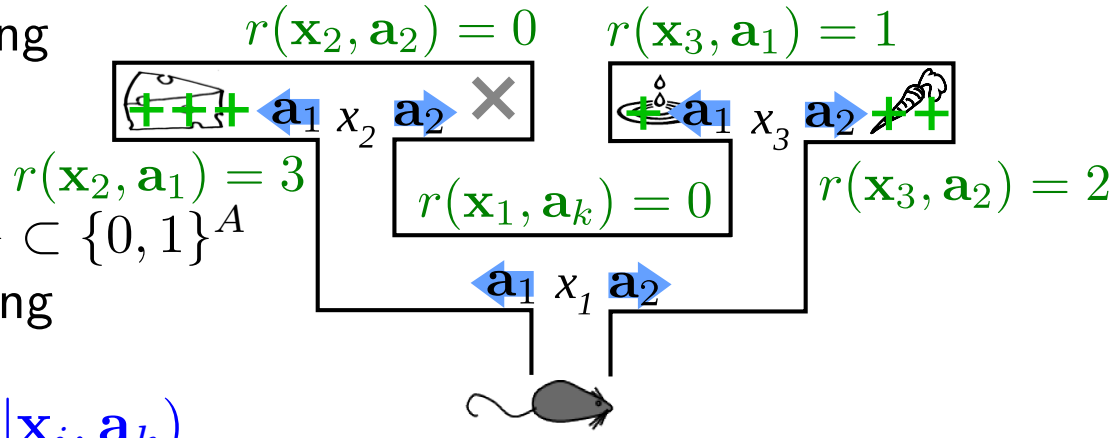


- a **transition model** $P(\underline{\mathbf{x}}_j | \underline{\mathbf{x}}_i, \underline{\mathbf{a}}_k)$
  - probability to end up in $\underline{\mathbf{x}}_j$ after choosing $\underline{\mathbf{a}}_k$ in $\underline{\mathbf{x}}_i$
  - stationary distribution (Markov property)

Markov decision processes

A Markov decision process (MDP) consists of

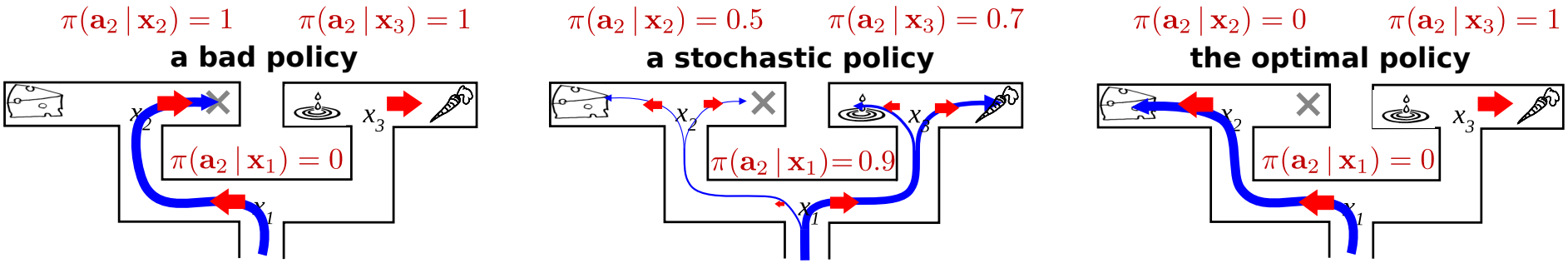
- a set of **states** $\underline{\mathbf{x}} \in \mathcal{X}$,
  - e.g. $\mathcal{X} := \{\underline{\mathbf{x}}_1, \ldots, \underline{\mathbf{x}}_S\} \subset \{0,1\}^S$ with 1-out-of-S encoding
- a set of **actions** $\underline{\mathbf{a}} \in \mathcal{A}$,
  - e.g. $\mathcal{A} := \{\underline{\mathbf{a}}_1, \ldots, \underline{\mathbf{a}}_A\} \subset \{0,1\}^A$ with 1-out-of-A encoding



- a **transition model** $P(\underline{\mathbf{x}}_j | \underline{\mathbf{x}}_i, \underline{\mathbf{a}}_k)$
  - probability to end up in $\underline{\mathbf{x}}_j$ after choosing $\underline{\mathbf{a}}_k$ in $\underline{\mathbf{x}}_i$
  - stationary distribution (Markov property)
- a bounded **reward function** $r(\underline{\mathbf{x}}_i, \underline{\mathbf{a}}_k)$
  - denotes the *immediate reward* for choosing $\underline{\mathbf{a}}_k$ in $\underline{\mathbf{x}}_i$
  - extension with randomized rewards possible

Policy

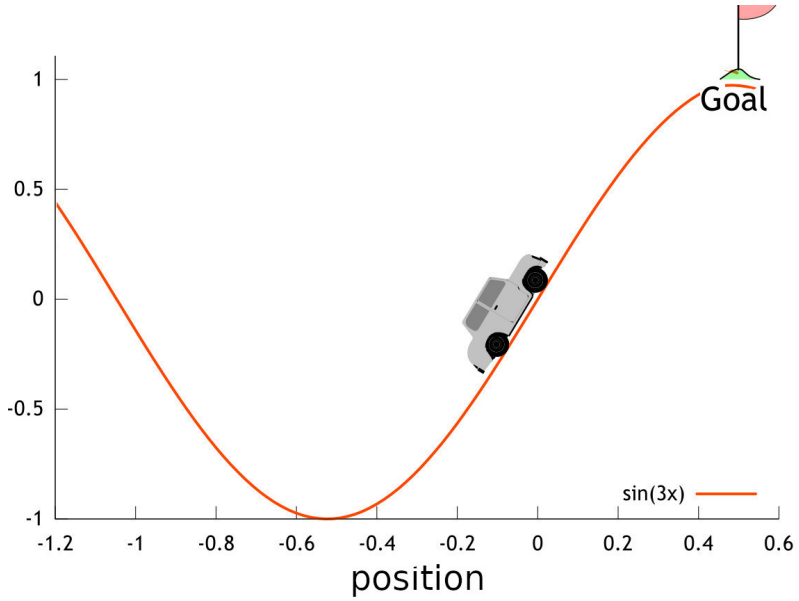
- The agent's behavior is expressed by a **policy** $\pi(\underline{\mathbf{a}}_k|\underline{\mathbf{x}}_i)$, i.e. the probability that the agent chooses $\underline{\mathbf{a}}_k$ in $\underline{\mathbf{x}}_i$.



- The goal of RL is to find the "**optimal policy**" π^*.

Example: mountain car

- a car in a valley between mountains
  - state space $\mathcal{X}$: position and velocity (discretized)
  - discretized time (intervals Δt)
- the car is driven by an agent
  - action space $\mathcal{A}$: no (0), forward (a) and backward $(-a)$ acceleration during time interval Δt
- dynamics is given by simplified physics
  - gravitation but no friction
  - transition model P is simulated
- goal: reach right hilltop
  - reward $r=0$, except $r=1$ at goal
- but: the car is underpowered
  - policy π must first pick up speed



Markov chains

- a **Markov chain** of length p
  - is a sequence of states and actions

$$\{\underline{\mathbf{x}}^{(t)}, \underline{\mathbf{a}}^{(t)}\}_{t=0}^{p} \quad \subset \quad \mathcal{X} \times \mathcal{A}$$

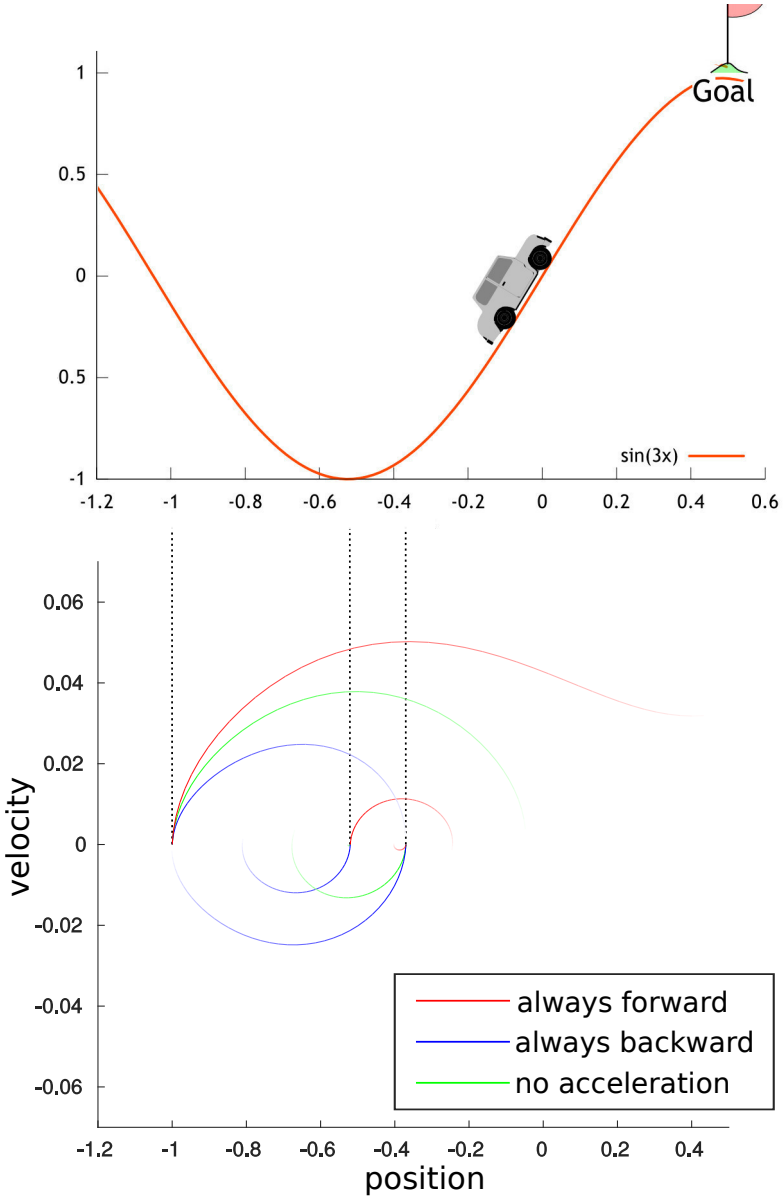
  - actions $\underline{\mathbf{a}}^{(t)}$ are drawn from the policy:

$$\underline{\mathbf{a}}^{(t)} \quad \sim \quad \pi(\cdot \,|\, \underline{\mathbf{x}}^{(t)})$$

  - successive states $\underline{\mathbf{x}}^{(t+1)}$ are drawn from the transition model:

$$\underline{\mathbf{x}}^{(t+1)} \quad \sim \quad P(\,\cdot \,|\, \underline{\mathbf{x}}^{(t)}, \underline{\mathbf{a}}^{(t)})$$

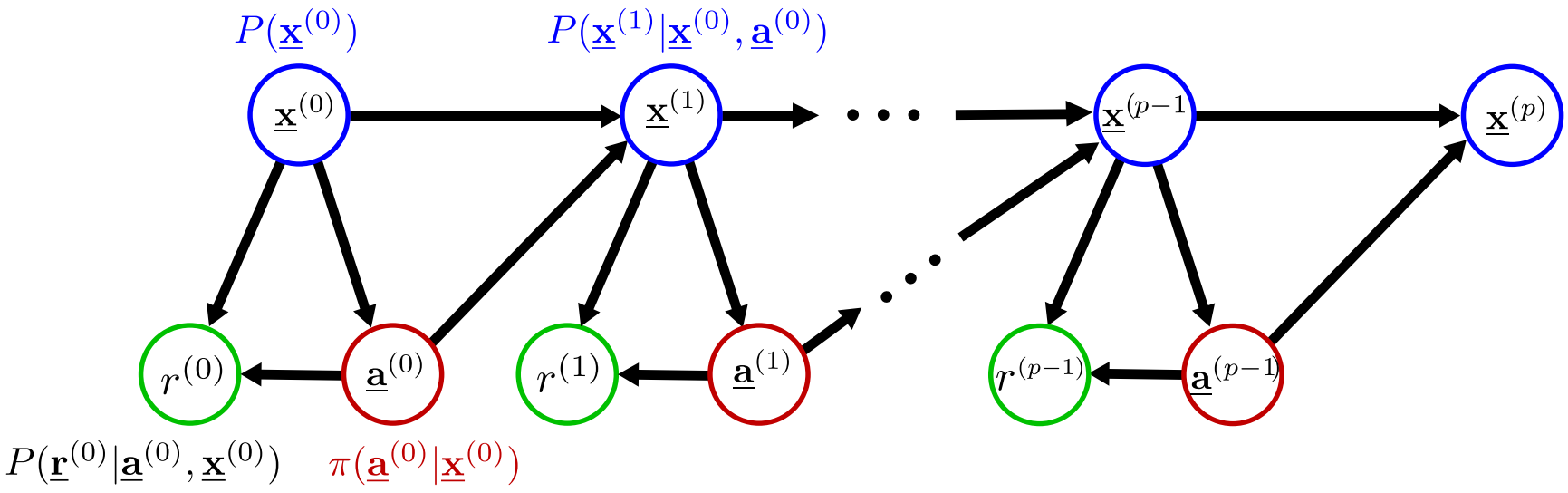
- given an MDP, the Markov chains depend on the initial state $\underline{\mathbf{x}}^{(0)}$ and the policy π



Markov chains

The joint distribution of states in a Markov chain factorizes.

$$P^\pi(\underline{\mathbf{x}}^{(0)}, \ldots, \underline{\mathbf{x}}^{(p)}) \quad = \quad P(\underline{\mathbf{x}}^{(0)}) \prod_{t=0}^{p-1} \sum_{k=1}^{A} \pi(\underline{\mathbf{a}}_k \,|\, \underline{\mathbf{x}}^{(t)})\, P(\underline{\mathbf{x}}^{(t+1)} | \, \underline{\mathbf{x}}^{(t)}, \underline{\mathbf{a}}_k)$$



(see blackboard)

4.1.2 Policy Evaluation

Value function

- a **value function** measures the quality of a policy π in state $\underline{\mathbf{x}}^{(0)}$
  - $V^\pi(\underline{\mathbf{x}}^{(0)})$ is the *expected* *reward*

$$V^\pi(\underline{\mathbf{x}}^{(0)}) = \mathbb{E}\left[r(\underline{\mathbf{x}}^{(0)}, \underline{\mathbf{a}}^{(0)}) \,\middle|\, \underline{\mathbf{a}}^{(0)} \sim \pi(\cdot \,|\, \underline{\mathbf{x}}^{(0)}) \right]$$

- averaged over all possible selected actions

Value function

- a **value function** measures the quality of a policy π in state $\underline{\mathbf{x}}^{(0)}$
  - $V^\pi(\underline{\mathbf{x}}^{(0)})$ is the *expected* *sum of* *future rewards*

$$V^\pi(\underline{\mathbf{x}}^{(0)}) = \mathbb{E}\left[\sum_{t=0}^{H} r(\underline{\mathbf{x}}^{(t)}, \underline{\mathbf{a}}^{(t)}) \;\middle|\; \begin{array}{c} \underline{\mathbf{a}}^{(t)} \sim \pi(\cdot \mid \underline{\mathbf{x}}^{(t)}) \\ \underline{\mathbf{x}}^{(t+1)} \sim P(\cdot \mid \underline{\mathbf{x}}^{(t)}, \underline{\mathbf{a}}^{(t)}) \end{array}\right]$$

- averaged over all possible Markov chains

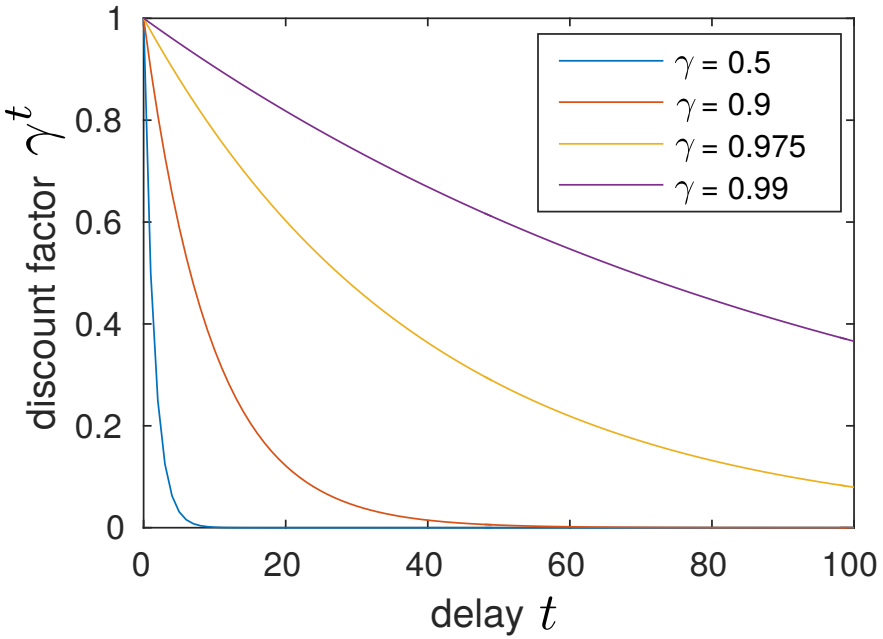
- each sample starts at $\underline{\mathbf{x}}^{(0)}$ and draws sequentially from the transition model P and the policy π

Value function

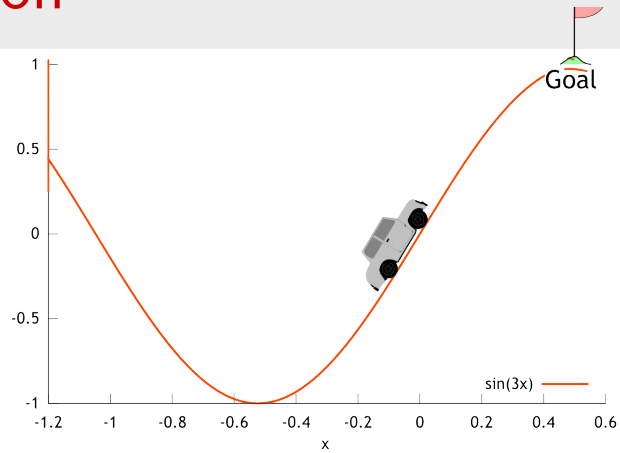
- a **value function** measures the quality of a policy π in state $\underline{\mathbf{x}}^{(0)}$
  - $V^\pi(\underline{\mathbf{x}}^{(0)})$ is the *expected infinite sum of discounted future rewards*

$$V^\pi(\underline{\mathbf{x}}^{(0)}) \;=\; \mathbb{E}\left[\sum_{t=0}^{\infty}\gamma^t\, r(\underline{\mathbf{x}}^{(t)},\underline{\mathbf{a}}^{(t)}) \;\middle|\; \begin{array}{c}\underline{\mathbf{a}}^{(t)} \sim \pi(\cdot\,|\,\underline{\mathbf{x}}^{(t)})\\ \underline{\mathbf{x}}^{(t+1)} \sim P(\cdot\,|\,\underline{\mathbf{x}}^{(t)},\underline{\mathbf{a}}^{(t)})\end{array}\right], \quad \gamma \in [0,1)\,.$$

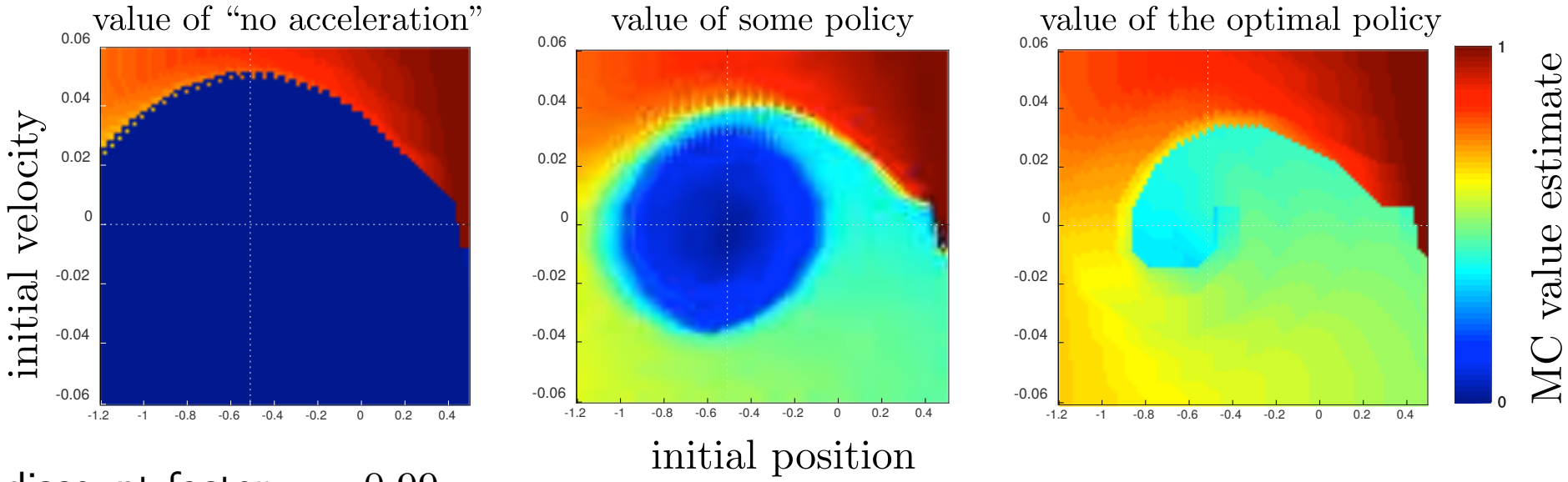
- averaged over all possible Markov chains
- each sample starts at $\underline{\mathbf{x}}^{(0)}$ and draws sequentially from the transition model P and the policy π
- discount factor γ: preference for short- vs. long-term goals



Monte Carlo (MC) estimation of the value function



- finite approximation of infinite Markov chains
  - rewards weighted by $\gamma^H < \epsilon$ are neglected
  - value is the discounted reward averaged over n Markov chains of length H
  - n must be sufficiently large
- requires our simulator to draw n chains from the same initial state
- every state must be evaluated often $\rightsquigarrow$ not sample efficient



discount factor $\gamma = 0.99$

The Bellman equation (1)

Richard E. Bellman
(1920–1984)

$$V^\pi(\underline{\mathbf{x}}_i) \quad = \quad \mathbb{E}\left[\sum_{t=0}^{\infty} \gamma^t \, r(\underline{\mathbf{x}}^{(t)}, \underline{\mathbf{a}}^{(t)}) \;\middle|\; \begin{array}{c} \underline{\mathbf{x}}^{(0)} := \underline{\mathbf{x}}_i \\ \underline{\mathbf{a}}^{(t)} \sim \pi(\cdot \,|\, \underline{\mathbf{x}}^{(t)}) \\ \underline{\mathbf{x}}^{(t+1)} \sim P(\cdot \,|\, \underline{\mathbf{x}}^{(t)}, \underline{\mathbf{a}}^{(t)}) \end{array}\right]$$

(see blackboard)

$\underline{\mathbf{x}}_i \in \{0,1\}^S$: 1-out-of-$S$ coded state i

The Bellman equation (1)

Richard E. Bellman
(1920–1984)

$$
\begin{aligned}
V^\pi(\underline{\mathbf{x}}_i) &= \mathbb{E}\left[\sum_{t=0}^{\infty} \gamma^t\, r(\underline{\mathbf{x}}^{(t)}, \underline{\mathbf{a}}^{(t)}) \;\middle|\; \begin{array}{c} \underline{\mathbf{x}}^{(0)} := \underline{\mathbf{x}}_i \\ \underline{\mathbf{a}}^{(t)} \sim \pi(\cdot \,|\, \underline{\mathbf{x}}^{(t)}) \\ \underline{\mathbf{x}}^{(t+1)} \sim P(\cdot \,|\, \underline{\mathbf{x}}^{(t)}, \underline{\mathbf{a}}^{(t)}) \end{array}\right] \\
&= \mathbb{E}\left[r(\underline{\mathbf{x}}_i, \underline{\mathbf{a}}^{(0)}) \;\middle|\; \underline{\mathbf{a}}^{(0)} \sim \pi(\cdot|\underline{\mathbf{x}}_i)\right] \\
&\quad + \mathbb{E}\left[\sum_{t=1}^{\infty} \gamma^t\, r(\underline{\mathbf{x}}^{(t)}, \underline{\mathbf{a}}^{(t)}) \;\middle|\; \begin{array}{c} \underline{\mathbf{x}}^{(0)} := \underline{\mathbf{x}}_i \\ \underline{\mathbf{a}}^{(t)} \sim \pi(\cdot \,|\, \underline{\mathbf{x}}^{(t)}) \\ \underline{\mathbf{x}}^{(t+1)} \sim P(\cdot \,|\, \underline{\mathbf{x}}^{(t)}, \underline{\mathbf{a}}^{(t)}) \end{array}\right] \\
&= \mathbb{E}\left[r(\underline{\mathbf{x}}_i, \underline{\mathbf{a}}^{(0)}) \;\middle|\; \underline{\mathbf{a}}^{(0)} \sim \pi(\cdot|\underline{\mathbf{x}}_i)\right] + \gamma\, \mathbb{E}\left[V^\pi(\underline{\mathbf{x}}^{(1)}) \;\middle|\; \begin{array}{c} \underline{\mathbf{a}}^{(0)} \sim \pi(\cdot \,|\, \underline{\mathbf{x}}_i) \\ \underline{\mathbf{x}}^{(1)} \sim P(\cdot|\underline{\mathbf{x}}_i, \underline{\mathbf{a}}^{(0)}) \end{array}\right] \\
&= \sum_{k=1}^{A} \pi(\underline{\mathbf{a}}_k \,|\, \underline{\mathbf{x}}_i) r(\underline{\mathbf{x}}_i, \underline{\mathbf{a}}_k) \;+\; \gamma \sum_{j=1}^{S} \sum_{k=1}^{A} \pi(\underline{\mathbf{a}}_k \,|\, \underline{\mathbf{x}}_i) P(\underline{\mathbf{x}}_j \,|\, \underline{\mathbf{x}}_i, \underline{\mathbf{a}}_k) V^\pi(\underline{\mathbf{x}}_j)
\end{aligned}
$$

$\underline{\mathbf{x}}_i \in \{0,1\}^S$: 1-out-of-$S$ coded state i

The Bellman equation (2)

$$V^\pi(\underline{\mathbf{x}}_i) = \underbrace{\sum_{k=1}^{A} \pi(\underline{\mathbf{a}}_k \,|\, \underline{\mathbf{x}}_i) r(\underline{\mathbf{x}}_i, \underline{\mathbf{a}}_k)}_{\text{“controlled” reward function } r_i^\pi} + \gamma \sum_{j=1}^{S} \underbrace{\sum_{k=1}^{A} \pi(\underline{\mathbf{a}}_k \,|\, \underline{\mathbf{x}}_i) P(\underline{\mathbf{x}}_j \,|\, \underline{\mathbf{x}}_i, \underline{\mathbf{a}}_k)}_{\text{“controlled” transition model } P_{ij}^\pi} V^\pi(\underline{\mathbf{x}}_j)$$

$$\begin{aligned} \underline{\mathbf{v}}^\pi &= \underline{\mathbf{r}}^\pi + \gamma \underline{\mathbf{P}}^\pi \underline{\mathbf{v}}^\pi \,, \quad \text{with} \\ &=: \hat{B}^\pi[\underline{\mathbf{v}}^\pi] \end{aligned} \quad \underbrace{\begin{cases} r_i^\pi := \sum_{k=1}^{A} \pi(\underline{\mathbf{a}}_k \,|\, \underline{\mathbf{x}}_i)\, r(\underline{\mathbf{x}}_i, \underline{\mathbf{a}}_k) \\ P_{ij}^\pi := \sum_{k=1}^{A} \pi(\underline{\mathbf{a}}_k \,|\, \underline{\mathbf{x}}_i)\, P(\underline{\mathbf{x}}_j | \underline{\mathbf{x}}_i, \underline{\mathbf{a}}_k) \end{cases}}_{\text{“controlled” models } \underline{\mathbf{r}}^\pi \in \mathbb{R}^S \text{ and } \underline{\mathbf{P}}^\pi \in \mathbb{R}^{S \times S}}$$

$\underline{\mathbf{x}}_i \in \{0,1\}^S$: 1-out-of-$S$ coded state i, $\underline{\mathbf{v}}^\pi \in \mathbb{R}^S$: vector containing all values V^π

4.1.3 Model-based Approaches

The analytical solution of the Bellman equation

Bellman equation:

$$\begin{aligned} \underline{\mathbf{v}}^\pi &= \underline{\mathbf{r}}^\pi + \gamma \underline{\mathbf{P}}^\pi \underline{\mathbf{v}}^\pi \\ (\underline{\mathbf{I}} - \gamma \underline{\mathbf{P}}^\pi) \underline{\mathbf{v}}^\pi &= \underline{\mathbf{r}}^\pi \\ \underline{\mathbf{v}}^\pi &= (\underline{\mathbf{I}} - \gamma \underline{\mathbf{P}}^\pi)^{-1} \underline{\mathbf{r}}^\pi \end{aligned}$$

- $|\lambda_k| \leq 1$ for all eigenvalues λ_k of the transition matrix $\underline{\mathbf{P}}^\pi$
- matrix $(\underline{\mathbf{I}} - \gamma \underline{\mathbf{P}}^\pi)$ has strictly positive eigenvalues for $\gamma < 1$
- matrix $(\underline{\mathbf{I}} - \gamma \underline{\mathbf{P}}^\pi) \in \mathbb{R}^{S \times S}$ is always invertible for $\gamma < 1$

(see e.g. Bertsekas, 2007, for details)

Solution by fixed point iteration

- the value function $\underline{\mathbf{v}}^\pi$ is the **fixed-point** of the Bellman operator $\hat{B}^\pi$

$$\underline{\mathbf{v}}^\pi \quad = \quad \hat{B}^\pi[\underline{\mathbf{v}}^\pi] \quad = \quad \underline{\mathbf{r}}^\pi + \gamma \underline{\mathbf{P}}^\pi \underline{\mathbf{v}}^\pi$$

- **value iteration**: repeated application of the Bellman operator

$$\underline{\tilde{\mathbf{v}}}^{\pi(t+1)} \quad = \quad \underline{\mathbf{r}}^\pi + \gamma \underline{\mathbf{P}}^\pi \underline{\tilde{\mathbf{v}}}^{\pi(t)}$$

$\Rightarrow$ value iteration converges in the limit $t \to \infty$

- number of iterations until convergence $\sim -\frac{1}{\log(\gamma)}$
- analytical solution is faster for large values of γ

to proof of convergence

4.1.4 Model-free Approaches: Online Value Estimation

Inductive value estimation

$$V^\pi(\underline{\mathbf{x}}_i) \;=\; \sum_{k=1}^{A} \pi(\underline{\mathbf{a}}_k \,|\, \underline{\mathbf{x}}_i) r(\underline{\mathbf{x}}_i, \underline{\mathbf{a}}_k) + \gamma \sum_{j=1}^{S} \sum_{k=1}^{A} \pi(\underline{\mathbf{a}}_k \,|\, \underline{\mathbf{x}}_i) P(\underline{\mathbf{x}}_j \,|\, \underline{\mathbf{x}}_i, \underline{\mathbf{a}}_k) V^\pi(\underline{\mathbf{x}}_j)$$

- reward, $r(\underline{\mathbf{x}}_i, \underline{\mathbf{a}}_k)$, and transition models, $P(\underline{\mathbf{x}}_j | \underline{\mathbf{x}}_i, \underline{\mathbf{a}}_k)$, may not be available to the agent
- estimate value function by sampling from one long Markov chain
  - actions $\underline{\mathbf{a}}^{(t)}$ are drawn according to the policy, $\underline{\mathbf{a}}^{(t)} \sim \pi(\cdot | \underline{\mathbf{x}}^{(t)})$,
  - which lead to transitions $\underline{\mathbf{x}}^{(t+1)} \sim P(\cdot | \underline{\mathbf{x}}^{(t)}, \underline{\mathbf{a}}^{(t)})$
  - and yield rewards $r^{(t)} := r(\underline{\mathbf{x}}^{(t)}, \underline{\mathbf{a}}^{(t)})$

Temporal difference (TD) learning

- asynchronous on-line update of the value function one state at a time

$$\tilde{V}^{\pi}_{(t+1)}(\underline{\mathbf{x}}^{(t)}) \;=\; \tilde{V}^{\pi}_{(t)}(\underline{\mathbf{x}}^{(t)}) \;+\; \eta\Big(\underbrace{r^{(t)} + \gamma\tilde{V}^{\pi}_{(t)}(\underline{\mathbf{x}}^{(t+1)}) - \tilde{V}^{\pi}_{(t)}(\underline{\mathbf{x}}^{(t)})}_{\text{TD-error } \Delta V_{(t)}}\Big)$$

  - model knowledge not required
  - named after the difference $\Delta V_{(t)}$ in value function estimates (TD-error)

- TD learning performs value iteration *on average*[1]
  - the expectation over all Markov chains that pass $\underline{\mathbf{x}}_i$ at time t reads $(\underline{\mathbf{x}}^{(t)} = \underline{\mathbf{x}}_i)$:

$$\underbrace{\mathbb{E}\big[\tilde{V}^{\pi}_{(t+1)}(\underline{\mathbf{x}}^{(t)})\big]}_{v_i^{\pi(t+1)}} \;=\; (1-\eta)\underbrace{\mathbb{E}\big[\tilde{V}^{\pi}_{(t)}(\underline{\mathbf{x}}^{(t)})\big]}_{v_i^{\pi(t)}} + \eta\Big(\underbrace{\mathbb{E}[r^{(t)}] + \gamma\mathbb{E}\big[\tilde{V}^{\pi}_{(t)}(\underline{\mathbf{x}}^{(t+1)})\big]}_{(\underline{\mathbf{r}}^{\pi}+\gamma\underline{\mathbf{P}}^{\pi}\underline{\mathbf{v}}^{\pi(t)})_i}\Big)$$

(see Sutton and Barto, 1998)

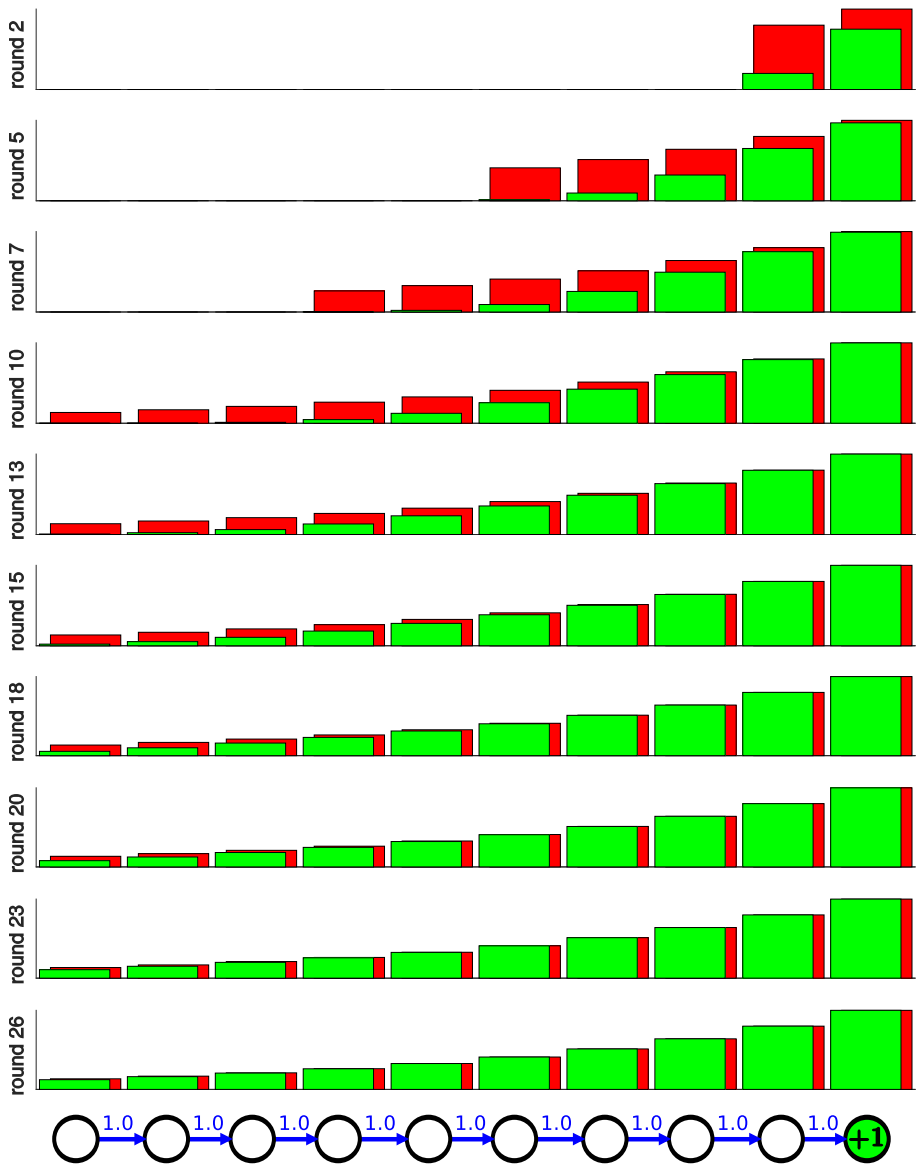
[1] for $\eta = 1$

Value propagation in TD learning

$$\tilde{V}^{\pi}_{(t+1)}(\underline{\mathbf{x}}^{(t)}) = \tilde{V}^{\pi}_{(t)}(\underline{\mathbf{x}}^{(t)}) + \eta\,\Delta V_{(t)}$$

$$\Delta V_{(t)} = r^{(t)} + \gamma \tilde{V}^{\pi}_{(t)}(\underline{\mathbf{x}}^{(t+1)}) - \tilde{V}^{\pi}_{(t)}(\underline{\mathbf{x}}^{(t)})$$

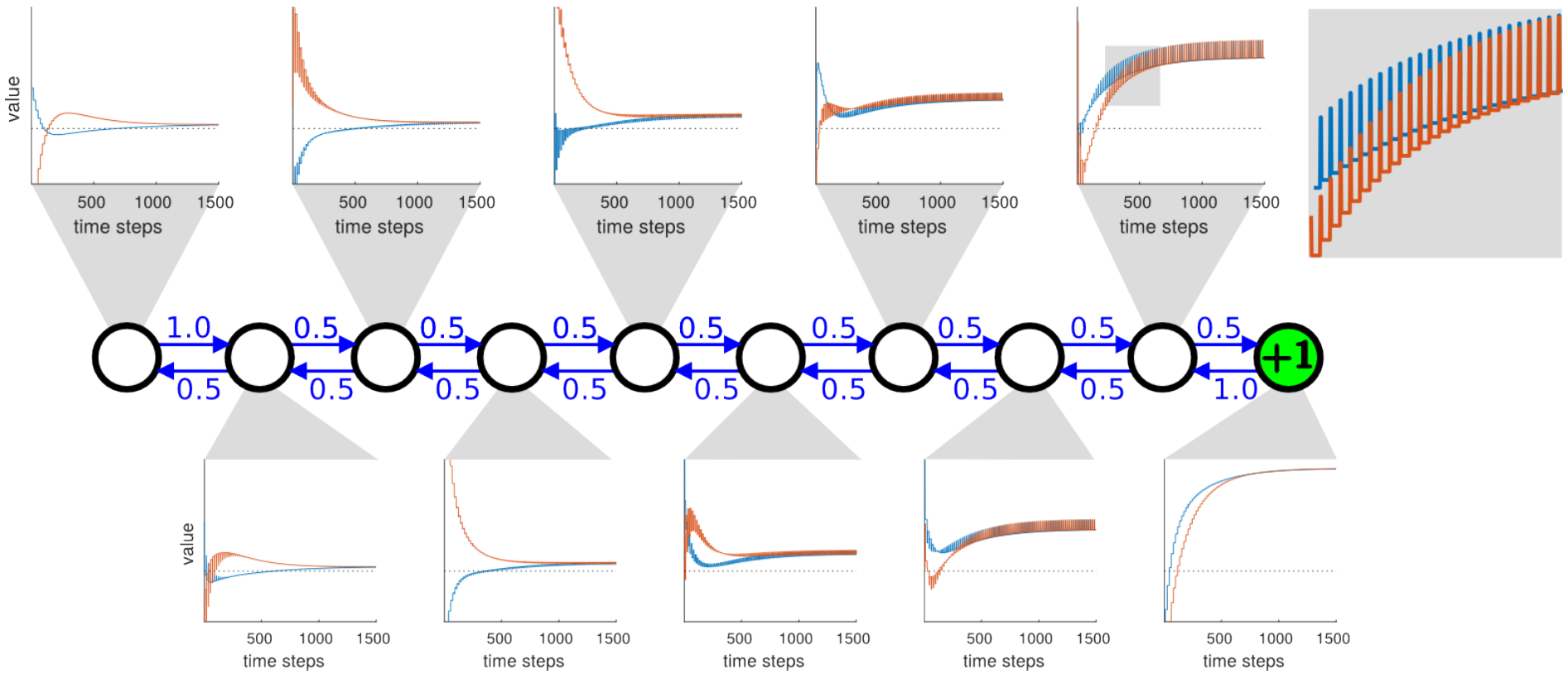
- TD learning propagates values one step into the past
- deterministic example:
  - 10 states, 1 action
  - only forward transitions
  - reward in last state
  - $\gamma = 0.9$; $\eta = 1$ or $\eta = 0.5$
  - value function initialized with 0
- value propagation requires
  - exactly 10 rounds ($\eta = 1$)
  - roughly 26 rounds ($\eta = 0.5$)



TD learning contracts but does not converge

Stochastic Markov chain with 10 states:

- two randomly initialized value functions (red/blue)
- deterministic transitions but stochastic policy
- rightmost state is rewarded, $\gamma = 0.95$, $\eta = 0.5$



Requirements for contraction

Ergodicity

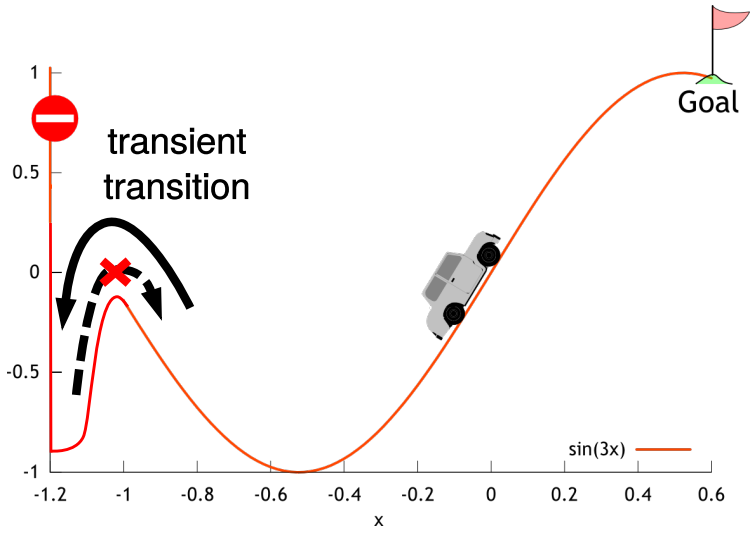
A Markov chain is **ergodic** if it is **positively recurrent** (non-zero probability to leave any state and eventually return to it) and **aperiodic** (returns to the same state occur at irregular times).

- **steady state distribution** $P_{\text{ss}}(\underline{\mathbf{x}}) > 0$: probability of the agent being in state $\underline{\mathbf{x}}$ is finite for all states $\underline{\mathbf{x}}$

$\Rightarrow$ TD learning contracts for *ergodic* Markov chains

$\Rightarrow$ but no convergence for constant $\eta > 0$ (contraction to neighborhood of true value)

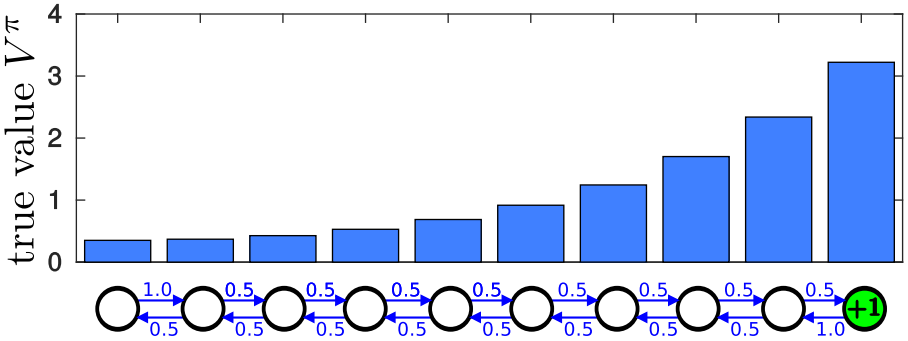
no **transient** transitions allowed



Influence of learning rate η

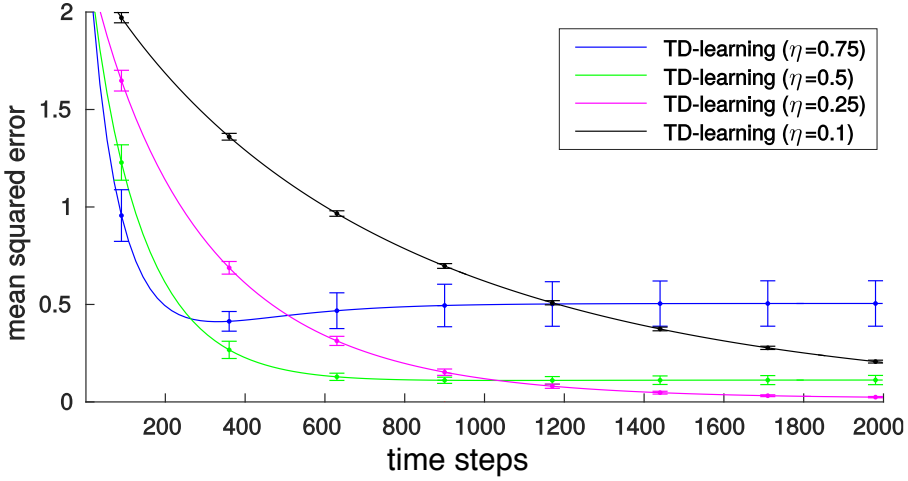
$$\tilde{V}^{\pi}_{(t+1)}(\underline{\mathbf{x}}^{(t)}) = \tilde{V}^{\pi}_{(t)}(\underline{\mathbf{x}}^{(t)}) + \eta\,\Delta V_{(t)}$$

$$\Delta V_{(t)} = r^{(t)} + \gamma \tilde{V}^{\pi}_{(t)}(\underline{\mathbf{x}}^{(t+1)}) - \tilde{V}^{\pi}_{(t)}(\underline{\mathbf{x}}^{(t)})$$



- stochastic transitions/policy/rewards $\rightsquigarrow \tilde{V}^{\pi}_{(t)}$ fluctuates around the true value V^{π}

- influence of the learning rate η
  - large η: fast learning, large variance
  - small η: slow learning, small variance
  - $\eta_{(t)}$ should decay, but finding a good annealing schedule may be difficult in practice



4.1.5 Model-free Approaches: Eligibility Traces & TD(λ)

Multi-step value propagation: Eligibility traces & TD(λ)

- **eligibility trace** $\underline{\mathbf{e}}^{(t)} \in \mathbb{R}^S$: Fading memory of past visits of state $\underline{\mathbf{x}}_i$.

$$
\begin{aligned}
e_i^{(t)} &= \sum_{k=0}^{t} (\gamma\lambda)^{t-k}\, \delta_{ik}\,, \qquad \delta_{ik} = \underline{\mathbf{x}}_i^\top \underline{\mathbf{x}}^{(k)} \qquad \forall \underline{\mathbf{x}}_i \in \mathcal{X}\,, \\
e_i^{(t+1)} &= \gamma\lambda e_i^{(t)} + \delta_{i,t+1} \qquad \text{on-line update}
\end{aligned}
$$

- The **TD(λ) method**: Add the current TD-error to **all** past states but scaled by the eligibility trace.

$$
\begin{aligned}
\tilde{V}^\pi_{(t+1)}(\underline{\mathbf{x}}_i) &= \tilde{V}^\pi_{(t)}(\underline{\mathbf{x}}_i) \;+\; \eta\, e_i{}^{(t)} \Big(\overbrace{r^{(t)} + \gamma \tilde{V}^\pi_{(t)}(\underline{\mathbf{x}}^{(t+1)}) - \tilde{V}^\pi_{(t)}(\underline{\mathbf{x}}^{(t)})}^{\text{TD-error } \Delta V_{(t)}} \Big) \\
\underline{\mathbf{e}}^{(t+1)} &= \gamma\, \lambda\, \underline{\mathbf{e}}^{(t)} \;+\; \underline{\mathbf{x}}^{(t+1)}
\end{aligned}
$$

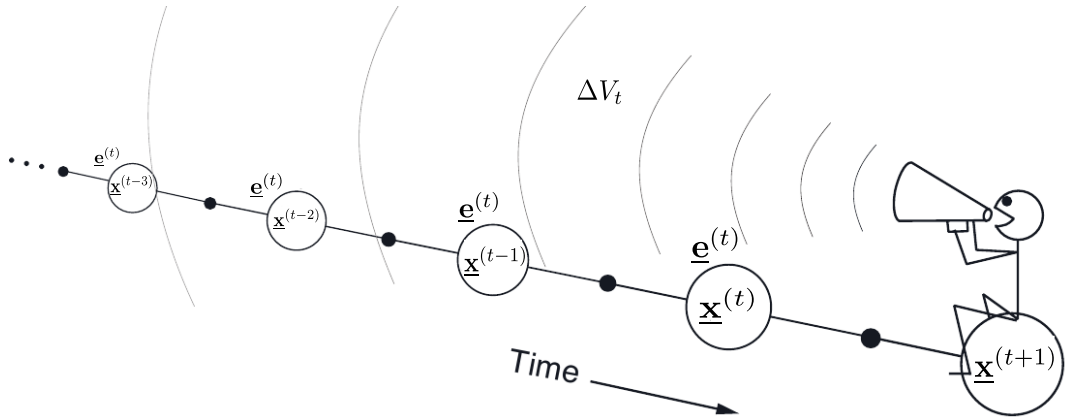
- **TD(0)**: TD learning as defined before

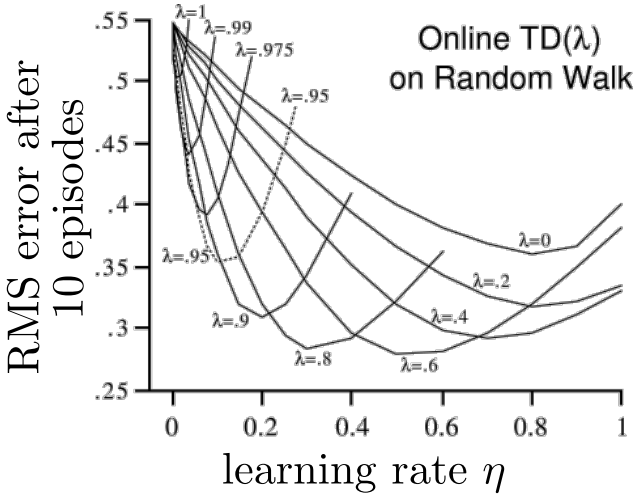
$$e_i^{(t)} = \delta_{ik} \rightarrow \text{only state } \underline{\mathbf{x}}^{(t)} \text{ changes its value}$$

Multi-step value propagation: Eligibility traces & TD(λ)

$$\tilde{V}^{\pi}_{(t+1)}(\underline{\mathbf{x}}_i) = \tilde{V}^{\pi}_{(t)}(\underline{\mathbf{x}}_i) + \eta\, e_i{}^{(t)} \overbrace{\left(r^{(t)} + \gamma \tilde{V}^{\pi}_{(t)}(\underline{\mathbf{x}}^{(t+1)}) - \tilde{V}^{\pi}_{(t)}(\underline{\mathbf{x}}^{(t)}) \right)}^{\text{TD-error } \Delta V_{(t)}}$$

$$\underline{\mathbf{e}}^{(t+1)} = \gamma\, \lambda\, \underline{\mathbf{e}}^{(t)} + \underline{\mathbf{x}}^{(t+1)}$$



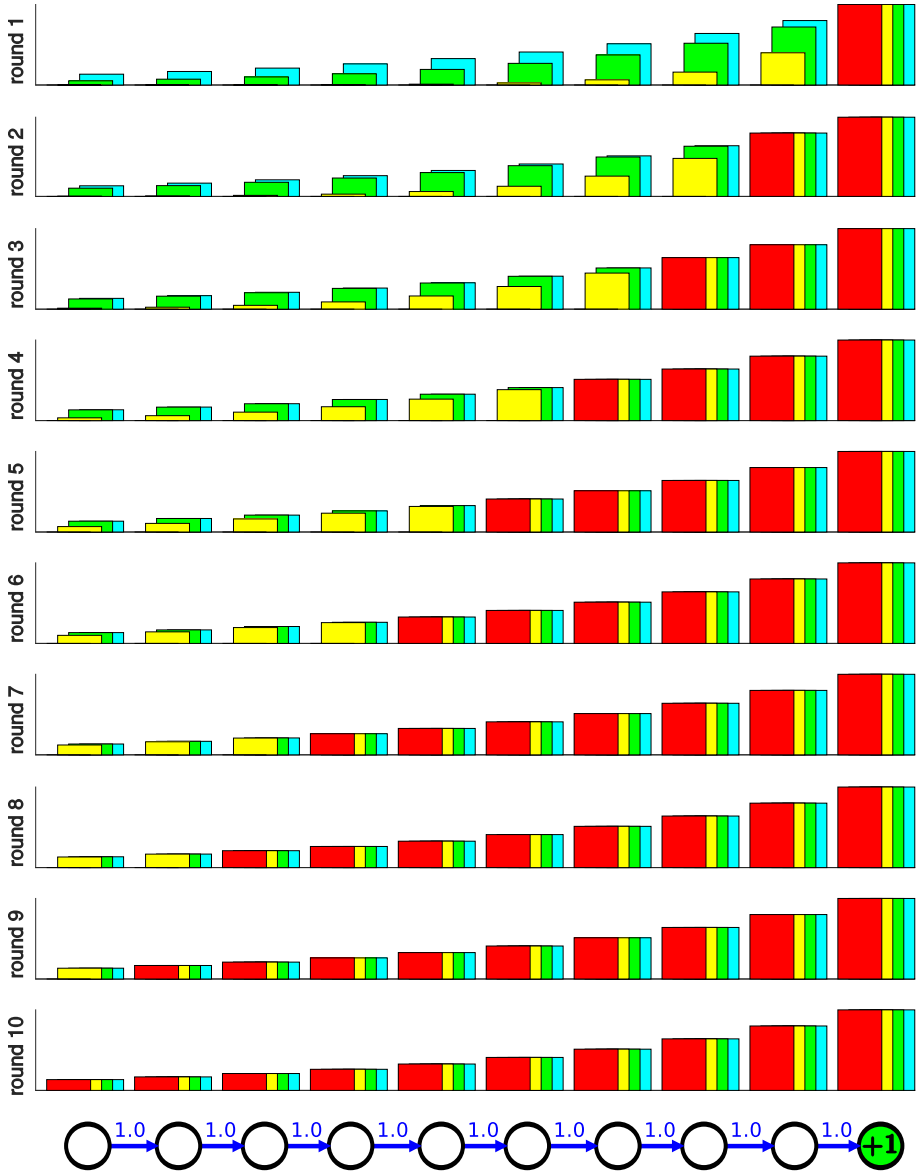


RMS averaged over 100 random-walks on a 19-state chain, rewarded at one end

(Sutton and Barto, 1998)

Value propagation in TD(λ)

- deterministic example:
  - 10 states, 1 action
  - only forward transitions
  - reward in last state
  - $\gamma = 0.9$, $\eta = 1$
- value propagation finishes
  - after 1 round with $\lambda = \mathbf{1}$
  - after 4 rounds with $\lambda = \mathbf{0.9}$
  - after 7 rounds with $\lambda = \mathbf{0.5}$
  - after 10 rounds with $\lambda = \mathbf{0}$



4.1.6 Model-free Approaches: Batch Value Estimation

Reminder: the Bellman equation

Richard E. Bellman
(1920–1984)

$$V^\pi(\underline{\mathbf{x}}_i) = \underbrace{\sum_{k=1}^{A} \pi(\underline{\mathbf{a}}_k \mid \underline{\mathbf{x}}_i) r(\underline{\mathbf{x}}_i, \underline{\mathbf{a}}_k)}_{\text{“controlled” reward function } r_i^\pi} + \gamma \sum_{j=1}^{S} \underbrace{\sum_{k=1}^{A} \pi(\underline{\mathbf{a}}_k \mid \underline{\mathbf{x}}_i) P(\underline{\mathbf{x}}_j \mid \underline{\mathbf{x}}_i, \underline{\mathbf{a}}_k)}_{\text{“controlled” transition model } P_{ij}^\pi} V^\pi(\underline{\mathbf{x}}_j)$$

$$\underline{\mathbf{v}}^\pi = \hat{B}^\pi[\underline{\mathbf{v}}^\pi] = \underline{\mathbf{r}}^\pi + \gamma \underline{\mathbf{P}}^\pi \underline{\mathbf{v}}^\pi$$

$\underline{\mathbf{x}}_i \in \{0,1\}^S$: 1-out-of-$S$ coded state i,

$\underline{\mathbf{r}}^\pi \in \mathbb{R}^S$ “controlled” reward function,

$\underline{\mathbf{v}}^\pi \in \mathbb{R}^S$: vector containing all values V^π

$\underline{\mathbf{P}}^\pi \in \mathbb{R}^{S \times S}$ “controlled” transition model

Batch approximation of the Bellman operator (1)

- estimate $\tilde{V}^{\pi}_{(t+1)} \approx \hat{B}^{\pi}[\tilde{V}^{\pi}_{(t)}]$ using samples $\{\underline{\mathbf{x}}^{(t)}, r^{(t)}, \underline{\mathbf{x}}^{(t+1)}\}_{t=0}^{p}$ from an ergodic Markov chain with executing policy π

$$\hat{B}^{\pi}[\tilde{V}^{\pi}_{s}](\underline{\mathbf{x}}_i) \quad = \quad \sum_{k=1}^{A} \pi(\underline{\mathbf{a}}_k \,|\, \underline{\mathbf{x}}_i) \Big(r(\underline{\mathbf{x}}_i, \underline{\mathbf{a}}_k) + \gamma \sum_{j=1}^{S} P(\underline{\mathbf{x}}_j \,|\, \underline{\mathbf{x}}_i, \underline{\mathbf{a}}_k)\, \tilde{V}^{\pi}_{s}(\underline{\mathbf{x}}_j) \Big)$$

Batch approximation of the Bellman operator (1)

- estimate $\tilde{V}^{\pi}_{(t+1)} \approx \hat{B}^{\pi}[\tilde{V}^{\pi}_{(t)}]$ using samples $\{\underline{\mathbf{x}}^{(t)}, r^{(t)}, \underline{\mathbf{x}}^{(t+1)}\}_{t=0}^{p}$ from an ergodic Markov chain with executing policy π

$$\hat{B}^{\pi}[\tilde{V}^{\pi}_{s}](\underline{\mathbf{x}}_i) = \sum_{k=1}^{A} \pi(\underline{\mathbf{a}}_k \,|\, \underline{\mathbf{x}}_i) \Big(r(\underline{\mathbf{x}}_i, \underline{\mathbf{a}}_k) + \gamma \sum_{j=1}^{S} P(\underline{\mathbf{x}}_j \,|\, \underline{\mathbf{x}}_i, \underline{\mathbf{a}}_k)\, \tilde{V}^{\pi}_{s}(\underline{\mathbf{x}}_j) \Big)$$

$$\approx \underbrace{\frac{1}{\sum_{\tau=0}^{p-1} \underline{\mathbf{x}}_i^{\top} \underline{\mathbf{x}}^{(\tau)}}}_{\text{normalization}} \sum_{t=0}^{p-1} \underbrace{\underline{\mathbf{x}}_i^{\top} \underline{\mathbf{x}}^{(t)}}_{\text{selection}} \Big(r^{(t)} + \gamma \tilde{V}^{\pi}_{(t)}(\underline{\mathbf{x}}^{(t+1)}) \Big)$$

- $\underline{\mathbf{x}}_i^{\top} \underline{\mathbf{x}}^{(t)} = 1$ only if $\underline{\mathbf{x}}_i = \underline{\mathbf{x}}^{(t)}$
- $\sum_{\tau=0}^{p-1} \underline{\mathbf{x}}_i^{\top} \underline{\mathbf{x}}^{(\tau)}$ counts how often $\underline{\mathbf{x}}_i$ appears in Markov chain

Batch approximation of the Bellman operator (2)

- estimate $\tilde{V}^\pi_{(t+1)} \approx \hat{B}^\pi[\tilde{V}^\pi_{(t)}]$ using samples $\{\underline{\mathbf{x}}^{(t)}, r^{(t)}, \underline{\mathbf{x}}^{(t+1)}\}_{t=0}^p$ from an ergodic Markov chain with executing policy π

$$\begin{aligned}
\hat{B}^\pi[\tilde{V}^\pi](\underline{\mathbf{x}}_i) &\approx \frac{1}{\sum_{\tau=0}^{p-1} \underline{\mathbf{x}}_i^\top \underline{\mathbf{x}}^{(\tau)}} \sum_{t=0}^{p-1} \underline{\mathbf{x}}_i^\top \underline{\mathbf{x}}^{(t)} \Big(r^{(t)} + \gamma \tilde{V}^\pi(\underline{\mathbf{x}}^{(t+1)}) \Big) \\
&= \underline{\mathbf{x}}_i^\top \underbrace{\Big(\sum_{\tau=0}^{p-1} \underline{\mathbf{x}}_i^\top \underline{\mathbf{x}}^{(\tau)} \Big)}_{C_{ii}}{}^{-1} \Big(\underbrace{\sum_{t=0}^{p-1} \underline{\mathbf{x}}^{(t)} r^{(t)}}_{\underline{\mathbf{b}}} + \gamma \underbrace{\sum_{t=0}^{p-1} \underline{\mathbf{x}}^{(t)} \tilde{V}^\pi(\underline{\mathbf{x}}^{(t+1)})}_{\underline{\mathbf{D}}^\pi \underline{\tilde{\mathbf{v}}}^\pi} \Big)
\end{aligned}$$

$$\hat{B}^\pi[\underline{\tilde{\mathbf{v}}}^\pi] \approx \underline{\mathbf{C}}^{-1} \big(\underline{\mathbf{b}} + \gamma \underline{\mathbf{D}}^\pi \underline{\tilde{\mathbf{v}}}^\pi \big)$$

$\underline{\mathbf{C}} = \sum_{t=0}^{p-1} \underline{\mathbf{x}}^{(t)} (\underline{\mathbf{x}}^{(t)})^\top \in \mathbb{R}^{S \times S}$ matrix of state visit counts

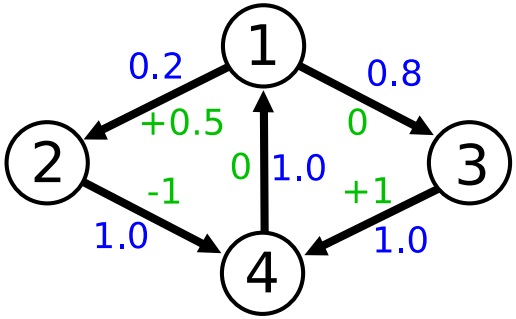
$\underline{\mathbf{D}}^\pi = \sum_{t=0}^{p-1} \underline{\mathbf{x}}^{(t)} (\underline{\mathbf{x}}^{(t+1)})^\top \in \mathbb{R}^{S \times S}$ matrix of transition counts

$\underline{\mathbf{b}} = \sum_{t=0}^{p-1} \underline{\mathbf{x}}^{(t)} r^{(t)} \in \mathbb{R}^S$ vector of sum of rewards

Batch value estimation: Example

- the approximated Bellman operator:

$$\hat{B}^\pi[\underline{\tilde{\mathbf{v}}}^\pi] \quad \approx \quad \underline{\mathbf{C}}^{-1}(\underline{\mathbf{b}} + \gamma \underline{\mathbf{D}}^\pi \underline{\tilde{\mathbf{v}}}^\pi)$$



- example MDP with 4 states:
  - example chain of length $p = 30$
  - transition probabilities $\underline{\mathbf{P}}^\pi \approx \underline{\mathbf{C}}^{-1}\underline{\mathbf{D}}^\pi$
  - reward for transitions $\underline{\mathbf{r}}^\pi \approx \underline{\mathbf{C}}^{-1}\underline{\mathbf{b}}$

$$\underline{\mathbf{C}} = \begin{bmatrix} 10 & 0 & 0 & 0 \\ 0 & 3 & 0 & 0 \\ 0 & 0 & 7 & 0 \\ 0 & 0 & 0 & 10 \end{bmatrix}, \qquad \underline{\mathbf{D}}^\pi = \begin{bmatrix} 0 & 3 & 7 & 0 \\ 0 & 0 & 0 & 3 \\ 0 & 0 & 0 & 7 \\ 10 & 0 & 0 & 0 \end{bmatrix}, \qquad \underline{\mathbf{b}} = \begin{bmatrix} 1.5 \\ -3 \\ +7 \\ 0 \end{bmatrix}$$

state visit counts | transition counts | collected rewards

$$\underline{\mathbf{C}} = \sum_{t=0}^{p-1} \underline{\mathbf{x}}^{(t)} (\underline{\mathbf{x}}^{(t)})^\top \in \mathbb{R}^{S \times S} \qquad \underline{\mathbf{D}}^\pi = \sum_{t=0}^{p-1} \underline{\mathbf{x}}^{(t)} (\underline{\mathbf{x}}^{(t+1)})^\top \in \mathbb{R}^{S \times S} \qquad \underline{\mathbf{b}} = \sum_{t=0}^{p-1} \underline{\mathbf{x}}^{(t)} r^{(t)} \in \mathbb{R}^S$$

Batch value estimation: Example

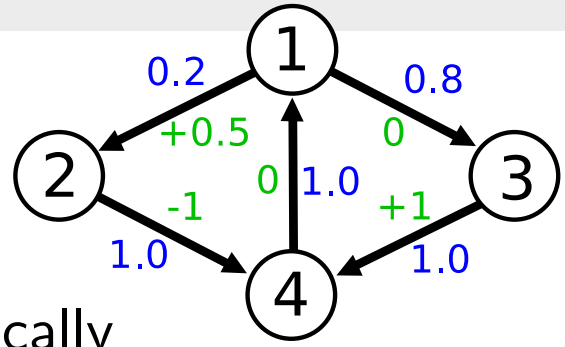
- the approximated Bellman operator:

$$\hat{B}^\pi[\underline{\tilde{\mathbf{v}}}^\pi] \quad \approx \quad \underline{\mathbf{C}}^{-1}(\underline{\mathbf{b}} + \gamma \underline{\mathbf{D}}^\pi \underline{\tilde{\mathbf{v}}}^\pi)$$

- fixed-point $\underline{\mathbf{v}}^* \approx \hat{B}^\pi[\underline{\mathbf{v}}^*]$ can be computed analytically

$$\underline{\mathbf{v}}^* = (\underline{\mathbf{I}} - \gamma \underbrace{\underline{\mathbf{C}}^{-1}\underline{\mathbf{D}}^\pi}_{\underline{\tilde{\mathbf{P}}}^\pi})^{-1} \underbrace{\underline{\mathbf{C}}^{-1}\underline{\mathbf{b}}}_{\underline{\tilde{\mathbf{r}}}^\pi} = (\underline{\mathbf{I}} - \gamma \underline{\tilde{\mathbf{P}}}^\pi)^{-1} \underline{\tilde{\mathbf{r}}}^\pi$$

Batch value estimation: Example



- the approximated Bellman operator:

$$\hat{B}^{\pi}[\underline{\tilde{\mathbf{v}}}^{\pi}] \quad \approx \quad \underline{\mathbf{C}}^{-1}(\underline{\mathbf{b}} + \gamma \underline{\mathbf{D}}^{\pi} \underline{\tilde{\mathbf{v}}}^{\pi})$$

- fixed-point $\underline{\mathbf{v}}^* \approx \hat{B}^{\pi}[\underline{\mathbf{v}}^*]$ can be computed analytically

$$\underline{\mathbf{v}}^* = (\underline{\mathbf{I}} - \gamma \underbrace{\underline{\mathbf{C}}^{-1}\underline{\mathbf{D}}^{\pi}}_{\underline{\tilde{\mathbf{P}}}^{\pi}})^{-1} \underbrace{\underline{\mathbf{C}}^{-1}\underline{\mathbf{b}}}_{\underline{\tilde{\mathbf{r}}}^{\pi}} = (\underline{\mathbf{I}} - \gamma \underline{\tilde{\mathbf{P}}}^{\pi})^{-1} \underline{\tilde{\mathbf{r}}}^{\pi}$$

- results for different values of γ for the example Markov chain

$\gamma = 0.95$

$$\underline{\mathbf{v}}^* = \begin{bmatrix} 3.7 \\ 2.4 \\ 4.4 \\ 3.5 \end{bmatrix}$$

$\gamma = 0.7$

$$\underline{\mathbf{v}}^* = \begin{bmatrix} 0.7 \\ -0.7 \\ 1.3 \\ 0.5 \end{bmatrix}$$

$\gamma = 0.3$

$$\underline{\mathbf{v}}^* = \begin{bmatrix} 0.3 \\ -1.0 \\ 1.0 \\ 0.1 \end{bmatrix}$$

Batch value estimation: Example

- the approximated Bellman operator:

$$\hat{B}^\pi[\underline{\tilde{\mathbf{v}}}^\pi] \quad \approx \quad \underline{\mathbf{C}}^{-1}(\underline{\mathbf{b}} + \gamma \underline{\mathbf{D}}^\pi \underline{\tilde{\mathbf{v}}}^\pi)$$

- fixed-point $\underline{\mathbf{v}}^* \approx \hat{B}^\pi[\underline{\mathbf{v}}^*]$ can be computed analytically

$$\underline{\mathbf{v}}^* = (\underline{\mathbf{I}} - \gamma \underbrace{\underline{\mathbf{C}}^{-1}\underline{\mathbf{D}}^\pi}_{\underline{\tilde{\mathbf{P}}}^\pi})^{-1} \underbrace{\underline{\mathbf{C}}^{-1}\underline{\mathbf{b}}}_{\underline{\tilde{\mathbf{r}}}^\pi} = (\underline{\mathbf{I}} - \gamma \underline{\tilde{\mathbf{P}}}^\pi)^{-1} \underline{\tilde{\mathbf{r}}}^\pi$$

- results for different values of γ for the example Markov chain

$\gamma = 0.95$

$$\underline{\mathbf{v}}^* = \begin{bmatrix} 3.7 \\ 2.4 \\ 4.4 \\ 3.5 \end{bmatrix}$$

$\gamma = 0.7$

$$\underline{\mathbf{v}}^* = \begin{bmatrix} 0.7 \\ -0.7 \\ 1.3 \\ 0.5 \end{bmatrix}$$

$\gamma = 0.3$

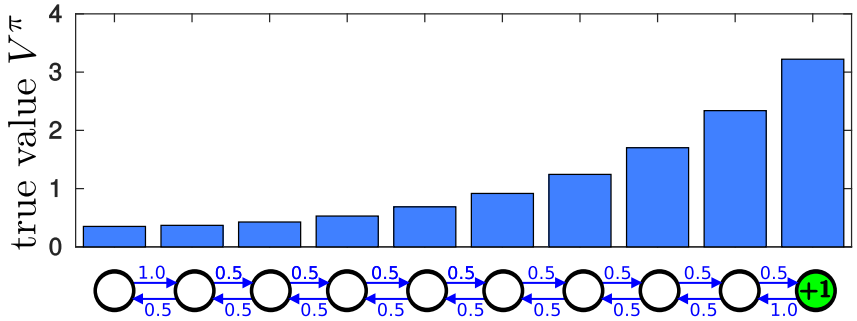
$$\underline{\mathbf{v}}^* = \begin{bmatrix} 0.3 \\ -1.0 \\ 1.0 \\ 0.1 \end{bmatrix}$$

- in the limit convergence to V^π for ergodic Markov chains
  - $\underline{\tilde{\mathbf{P}}}^\pi \to \underline{\mathbf{P}}^\pi$ and $\underline{\tilde{\mathbf{r}}}^\pi \to \underline{\mathbf{r}}^\pi$ if all states are visited infinitely often

Comparison of batch and online value estimation

- reward propagation

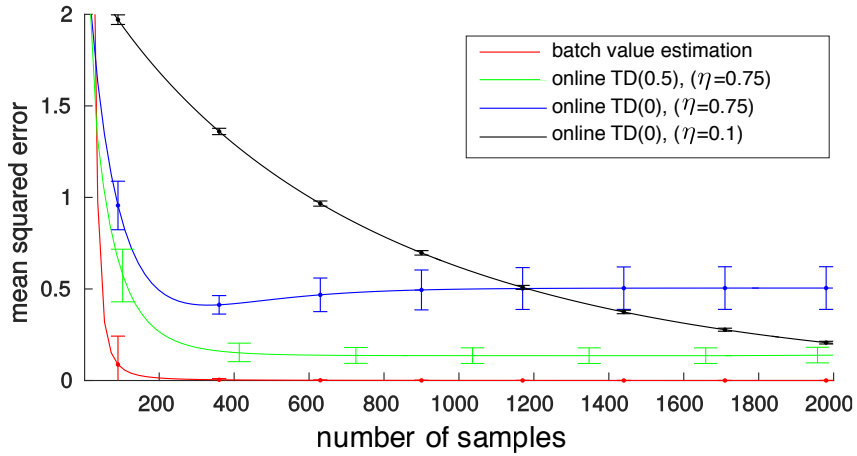
TD(0): one time step into the past
TD(λ): all (λ-discounted) steps into the past
batch: fixed-point computation (no propagation), no. of steps $\sim -\frac{1}{\log(\gamma)}$



- convergence behavior

TD(0): fluctuates around V^π
TD(λ): fluctuates around V^π
batch: converges to V^π

- Markov chain back and forth
- only last state rewarded
- value estimated for $\gamma = 0.95$



- computational complexity
  (time and memory)

TD(0): $\mathcal{O}(p)$ and $\mathcal{O}(S)$
TD(λ): $\mathcal{O}(pS)$ and $\mathcal{O}(S)$
batch: $\mathcal{O}(p + S^3)$ and $\mathcal{O}(S^2)$

S: number of states, p: number of samples

End of Section 4.1

the following slides contain

OPTIONAL MATERIAL

Convergence of value iteration

Contraction mapping (in supremum norm)

A function $\hat{B} : \mathbb{R}^S \to \mathbb{R}^S$ is called a *contraction mapping* with Lipschitz constant $0 \leq \lambda < 1$ if $\max_j \left|(\hat{B}[\underline{\tilde{\mathbf{v}}}] - \hat{B}[\underline{\tilde{\mathbf{w}}}])_j\right| \;\leq\; \lambda \max_j \left|\tilde{v}_j - \tilde{w}_j\right|, \forall \underline{\tilde{\mathbf{v}}}, \underline{\tilde{\mathbf{w}}} \in \mathbb{R}^S$.

- application to the Bellman operator $\hat{B}^\pi[\underline{\tilde{\mathbf{v}}}] = \underline{\mathbf{r}}^\pi + \gamma \underline{\mathbf{P}}^\pi \underline{\tilde{\mathbf{v}}}$

$$\begin{aligned}
\max_j \left|\hat{B}^\pi[\underline{\tilde{\mathbf{v}}}]_j - \hat{B}^\pi[\underline{\tilde{\mathbf{w}}}]_j\right| &= \max_j \left|r_j^\pi + \gamma(\underline{\mathbf{P}}^\pi \underline{\tilde{\mathbf{v}}})_j - r_j^\pi - \gamma(\underline{\mathbf{P}}^\pi \underline{\tilde{\mathbf{w}}})_j\right| \\
&\overset{\text{(i)}}{\leq} \max_j \gamma \left(\underline{\mathbf{P}}^\pi \left|\underline{\tilde{\mathbf{v}}} - \underline{\tilde{\mathbf{w}}}\right|\right)_j \overset{\text{(ii)}}{\leq} \gamma \max_j \left|\tilde{v}_j - \tilde{w}_j\right|
\end{aligned}$$

$$\text{(i)} \quad \left|\sum_{i=1}^S P_{ji}^\pi x_i\right| \leq \sum_{i=1}^S P_{ji}^\pi \left|x_i\right|, \qquad \forall \underline{\mathbf{x}} \in \mathbb{R}^S \qquad \text{(Jensen's inequality)}$$

$$\text{(ii)} \quad \sum_{i=1}^S P_{ji}^\pi \left|x_i\right| \leq \sum_{i=1}^S P_{ji}^\pi \max_{1 \leq k \leq S} \left|x_k\right| = \max_{1 \leq k \leq S} \left|x_k\right| \qquad \left(\sum_{i=1}^S P_{ji}^\pi = 1\right)$$

Convergence of value iteration

Contraction mapping (in supremum norm)

A function $\hat{B} : \mathbb{R}^S \to \mathbb{R}^S$ is called a *contraction mapping* with Lipschitz constant $0 \leq \lambda < 1$ if $\max_j \left|(\hat{B}[\underline{\tilde{\mathbf{v}}}] - \hat{B}[\underline{\tilde{\mathbf{w}}}])_j\right| \;\leq\; \lambda \max_j \left|\tilde{v}_j - \tilde{w}_j\right|, \forall \underline{\tilde{\mathbf{v}}}, \underline{\tilde{\mathbf{w}}} \in \mathbb{R}^S$.

- $\hat{B}^\pi$ is a contraction mapping with Lipschitz constant $0 < \gamma < 1$

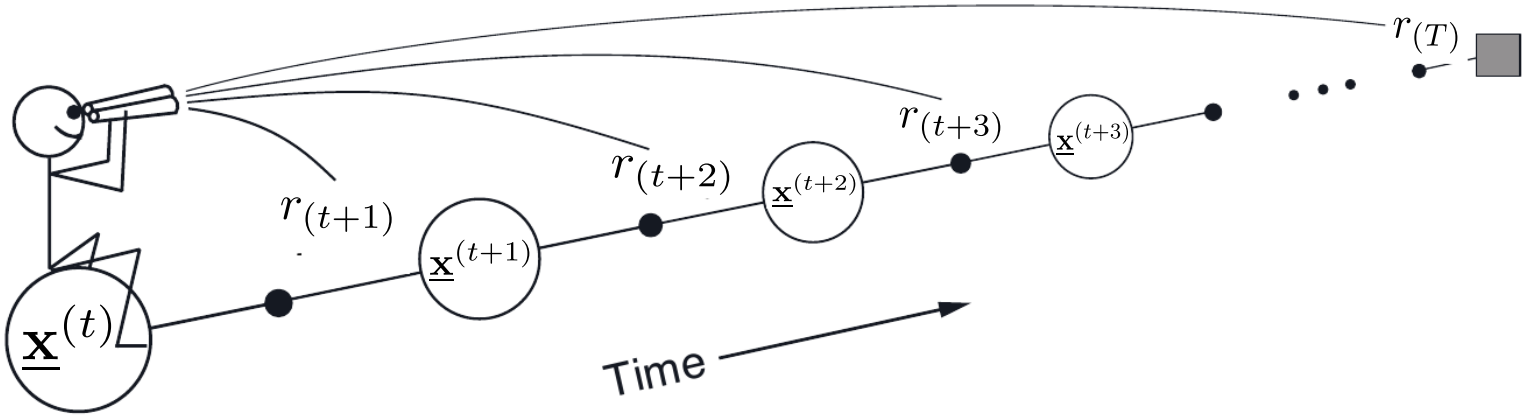
$$\underline{\tilde{\mathbf{v}}}^{\pi(t+1)} \quad = \quad \hat{B}^\pi[\underline{\tilde{\mathbf{v}}}^{\pi(t)}] \quad = \quad \underline{\mathbf{r}}^\pi + \gamma \underline{\mathbf{P}}^\pi \underline{\tilde{\mathbf{v}}}^{\pi(t)} \qquad \text{(value iteration)}$$

- Let $\underline{\mathbf{v}}^\pi$ be the unique fixed point. Then:

$$\max_j \left|(\hat{B}^\pi[\underline{\mathbf{v}}^{\pi(t)}])_j - (\hat{B}^\pi[\underline{\mathbf{v}}^\pi])_j\right| \leq \gamma \max_j \left|v_j^{\pi(t)} - v_j^\pi\right| \text{ for } \textit{any } \underline{\mathbf{v}}^{\pi(t)} \in \mathbb{R}^S$$

Relationship of TD(λ) to n-step TD-learning

$$
\begin{aligned}
R_{(t)}^{(n)} &= \sum_{\tau=0}^{n-1} \gamma^{\tau} r^{(t+\tau)} + \gamma^{n} \tilde{V}_{(t)}^{\pi}(\underline{\mathbf{x}}^{(t+n)}) \\
\tilde{V}_{(t+1)}^{\pi}(\underline{\mathbf{x}}^{(t)}) &= \tilde{V}_{(t)}^{\pi}(\underline{\mathbf{x}}^{(t)}) + \eta\Big(R_{(t)}^{(n)} - \tilde{V}_{(t)}^{\pi}(\underline{\mathbf{x}}^{(t)})\Big) \\
R_{(t)}^{\lambda} &= (1-\lambda) \sum_{k=0}^{\infty} \lambda^{k} R_{(t)}^{(k+1)} \\
\tilde{V}_{(t+1)}^{\pi}(\underline{\mathbf{x}}^{(t)}) &= \tilde{V}_{(t)}^{\pi}(\underline{\mathbf{x}}^{(t)}) + \eta\Big(R_{(t)}^{\lambda} - \tilde{V}_{(t)}^{\pi}(\underline{\mathbf{x}}^{(t)})\Big)
\end{aligned}
$$

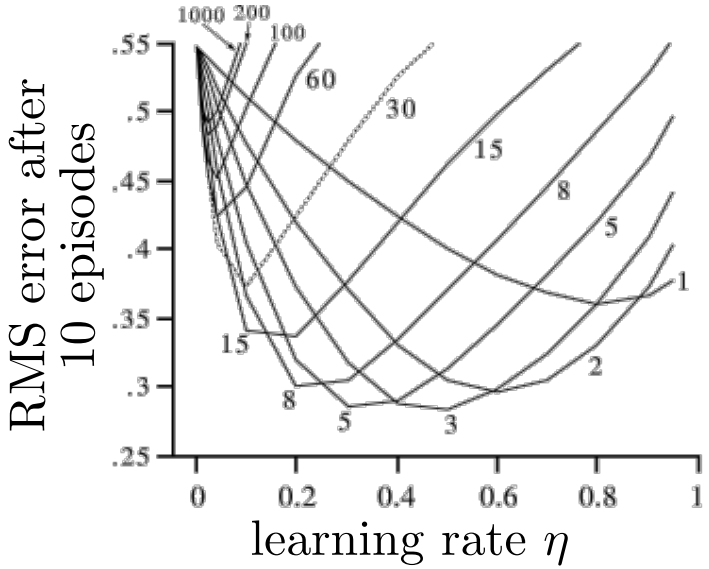


derivation here

Speeding up value propagation: n-step TD-learning

- accumulation of observed rewards

$$
\begin{aligned}
R^{(1)}_{(t)} &= r^{(t)} + \gamma \tilde{V}^{\pi}_{(t)}(\underline{\mathbf{x}}^{(t+1)}) \\
R^{(2)}_{(t)} &= r^{(t)} + \gamma r^{(t+1)} + \gamma^2 \tilde{V}^{\pi}_{(t)}(\underline{\mathbf{x}}^{(t+2)}) \\
&\vdots \\
R^{(n)}_{(t)} &= \sum_{\tau=0}^{n-1} \gamma^{\tau} r^{(t+\tau)} + \gamma^n \tilde{V}^{\pi}_{(t)}(\underline{\mathbf{x}}^{(t+n)})
\end{aligned}
$$



RMS error averaged over 100 random-walks on a 19-state chain, rewarded at one end

(Sutton and Barto, 1998)

- online estimation similar to TD learning

$$
\tilde{V}^{\pi}_{(t+1)}(\underline{\mathbf{x}}^{(t)}) \quad \leftarrow \quad \tilde{V}^{\pi}_{(t)}(\underline{\mathbf{x}}^{(t)}) \; + \; \eta\Big(R^{(n)}_{(t)} - \tilde{V}^{\pi}_{(t)}(\underline{\mathbf{x}}^{(t)})\Big)
$$

Discounted average

$$\tilde{V}^{\pi}_{(t+1)}(\underline{\mathbf{x}}^{(t)}) = \tilde{V}^{\pi}_{(t)}(\underline{\mathbf{x}}^{(t)}) + \eta\Big(R^{(n)}_{(t)} - \tilde{V}^{\pi}_{(t)}(\underline{\mathbf{x}}^{(t)})\Big)$$

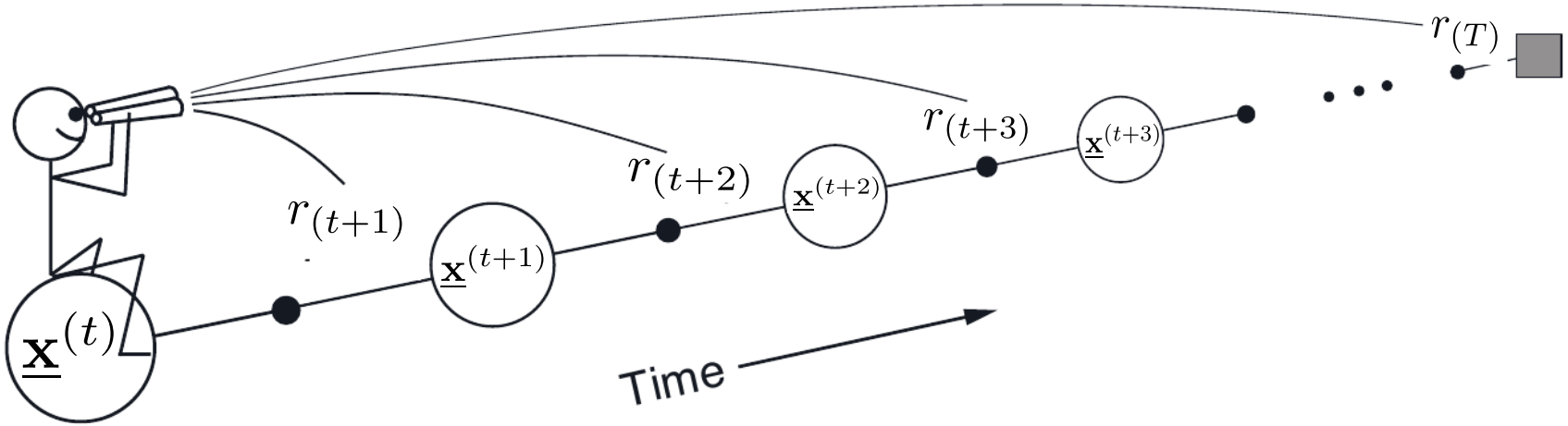
$$R^{(n)}_{(t)} = \sum_{\tau=0}^{n-1} \gamma^{\tau} r^{(t+\tau)} + \gamma^{n} \tilde{V}^{\pi}_{(t)}(\underline{\mathbf{x}}^{(t+n)})$$

- there is an optimal combination of η and n, however,
  - agent must memorize the last n steps
  - values are updated with a delay of n steps

- trick: consider a discounted average of $R^{(n)}_{(t)}$

$$R^{\lambda}_{(t)} = (1-\lambda) \sum_{k=0}^{\infty} \lambda^{k} R^{(k+1)}_{(t)}$$

Forward view

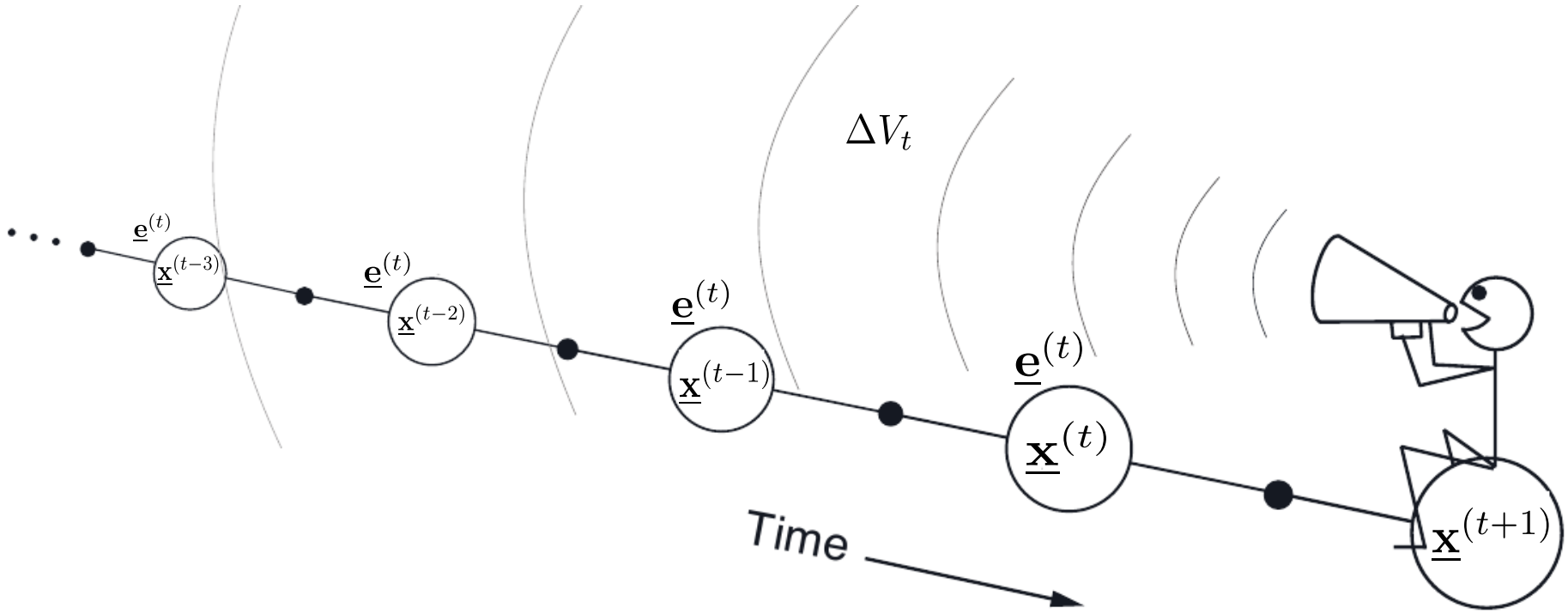


$$\tilde{V}^F_{(t+1)}(\underline{\mathbf{x}}^{(t)}) = \tilde{V}^F_{(t)}(\underline{\mathbf{x}}^{(t)}) + \eta\Big(R^\lambda_{(t)} - \tilde{V}^F_{(t)}(\underline{\mathbf{x}}^{(t)})\Big)$$

$$R^\lambda_{(t)} = (1-\lambda)\sum_{k=0}^{\infty}\lambda^k\, R^{(k+1)}_{(t)}$$

$$R^{(k)}_{(t)} = \sum_{\tau=0}^{k-1}\gamma^\tau r^{(t+\tau)} + \gamma^k \tilde{V}^\pi_{(t)}(\underline{\mathbf{x}}^{(t+k)})$$

Backward view



$$\tilde{V}^B_{(t+1)}(\underline{\mathbf{x}}_i) = \tilde{V}^B_{(t)}(\underline{\mathbf{x}}_i) + \eta e_i^{(t)} \Delta V_{(t)}, \quad \Delta V_t = (r^{(t)} + \gamma \tilde{V}^B_{(t)}(\underline{\mathbf{x}}^{(t+1)}) - \tilde{V}^B_{(t)}(\underline{\mathbf{x}}^{(t)}))$$

$$\underline{\mathbf{e}}^{(t+1)} = \gamma\lambda\underline{\mathbf{e}}^{(t)} + \underline{\mathbf{x}}^{(t+1)}$$

- Both the forward and the backward view updates are *equivalent*

TD(λ): forward vs. backward view

- we use $(1-\lambda)\sum_{i=0}^{\infty}\lambda^i = 1$ and $\sum_{i=0}^{n}\sum_{j=0}^{i} A_{ij} = \sum_{j=0}^{n}\sum_{i=j}^{n} A_{ij}$
- The forward value at time t is called $V^F_{(t)}$, the backward value $V^B_{(t)}$
- the TD-error at time t is $\Delta V_{(t)} = r^{(t)} + \gamma V^{F/B}_{(t)}(\underline{\mathbf{x}}^{(t+1)}) - V^{F/B}_{(t)}(\underline{\mathbf{x}}^{(t)})$
- initialization of the value function with 0 (without loss of generality)

TD(λ): the forward view

$$
\begin{aligned}
R^{\lambda}_{(t)} - V^F_{(t)}(\underline{\mathbf{x}}^{(t)}) &= (1-\lambda)\sum_{k=0}^{\infty}\lambda^k R^{(k+1)}_{(t)} - V^F_{(t)}(\underline{\mathbf{x}}^{(t)}) \\
&= (1-\lambda)\sum_{k=0}^{\infty}\lambda^k\left(\sum_{\tau=0}^{k}\gamma^\tau r^{(t+\tau)} + \gamma^{k+1}V^F_{(t)}(\underline{\mathbf{x}}^{(t+k+1)})\right) - V^F_{(t)}(\underline{\mathbf{x}}^{(t)}) \\
&= (1-\lambda)\sum_{\tau=0}^{\infty}\sum_{k=\tau}^{\infty}\lambda^k\gamma^\tau r^{(t+\tau)} + \gamma(1-\lambda)\sum_{k=0}^{\infty}\lambda^k\gamma^k V^F_{(t)}(\underline{\mathbf{x}}^{(t+k+1)}) - V^F_{(t)}(\underline{\mathbf{x}}^{(t)}) \\
&= \sum_{\tau=0}^{\infty}\gamma^\tau r^{(t+\tau)}\,\lambda^\tau\left[(1-\lambda)\sum_{k=0}^{\infty}\lambda^k\right] + \gamma(1-\lambda)\sum_{k=0}^{\infty}(\gamma\lambda)^k V^F_{(t)}(\underline{\mathbf{x}}^{(t+k+1)}) - V^F_{(t)}(\underline{\mathbf{x}}^{(t)}) \\
&= \sum_{k=0}^{\infty}(\gamma\lambda)^k\left(r^{(t+k)} + \gamma V^F_{(t)}(\underline{\mathbf{x}}^{(t+k+1)}) - \gamma\lambda\, V^F_{(t)}(\underline{\mathbf{x}}^{(t+k+1)})\right) - V^F_{(t)}(\underline{\mathbf{x}}^{(t)}) \\
&= \sum_{k=0}^{\infty}(\gamma\lambda)^k\left(r^{(t+k)} + \gamma V^F_{(t)}(\underline{\mathbf{x}}^{(t+k+1)}) - V^F_{(t)}(\underline{\mathbf{x}}^{(t+k)})\right) \\
&\approx \sum_{k=0}^{\infty}(\gamma\lambda)^k\,\Delta V_{(t+k)} \qquad \text{(approximation is good for stationary } V^F_{(t)}\text{)} \\
&\approx \sum_{k=t}^{T-1}(\gamma\lambda)^{k-t}\,\Delta V_{(k)} \qquad \text{(approximation is good for large } T\text{)}
\end{aligned}
$$

TD(λ): the backward view

$$
\begin{aligned}
V_T^B(\underline{\mathbf{x}}_i) &= \sum_{t=0}^{T-1} \eta \; \Delta V_{(t)} \; e_i^{(t)} \quad = \quad \eta \sum_{t=0}^{T-1} \Delta V_{(t)} \sum_{k=0}^{t} (\gamma\lambda)^{t-k} \, \delta_{ik} \\
&= \eta \sum_{k=0}^{T-1} \delta_{ik} \sum_{t=k}^{T-1} (\gamma\lambda)^{t-k} \, \Delta V_{(t)} \\
&= \eta \sum_{t=0}^{T-1} \delta_{it} \sum_{k=t}^{T-1} (\gamma\lambda)^{k-t} \, \Delta V_{(k)}
\end{aligned}
$$

TD(λ): forward vs. backward view

- the TD(λ) value of state $\underline{\mathbf{x}}_i$ at time T is

$$V^B_{(T)}(\underline{\mathbf{x}}_i) \quad = \quad \eta \sum_{t=0}^{T-1} \delta_{it} \sum_{k=t}^{T-1} (\gamma\lambda)^{k-t} \, \Delta V_{(k)}$$

- the value of state $\underline{\mathbf{x}}_i$ at time T in the forward view is

$$V^F_{(T)}(\underline{\mathbf{x}}_i) \quad = \quad \sum_{t=0}^{T-1} \eta \, \delta_{it} \big(R^\lambda_{(t)} - V^F_{(t)}(\underline{\mathbf{x}}^{(t)}) \big) \quad \approx \quad \eta \sum_{t=0}^{T-1} \delta_{it} \sum_{k=t}^{T-1} (\gamma\lambda)^{k-t} \, \Delta V_{(k)}$$

- in the limit of infinite training samples the approximation becomes exact

$$V^F_{(\infty)}(\underline{\mathbf{x}}_i) \quad = \quad V^B_{(\infty)}(\underline{\mathbf{x}}_i)$$

Temporal difference learning with eligibility traces: TD(λ)

The TD(λ) algorithm

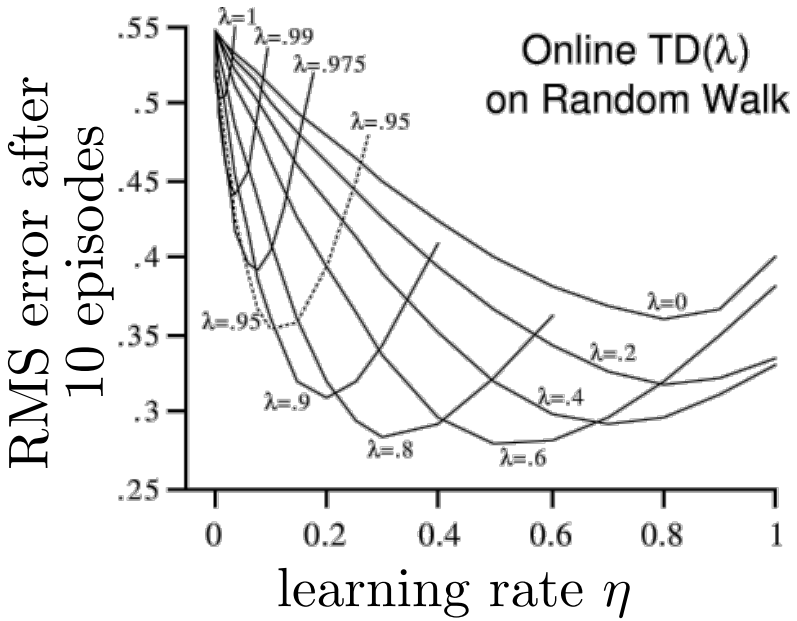
for $t \in \{0, \ldots, p-1\}$ **do**

$$\Delta v \leftarrow r^{(t)} + \gamma \underline{\mathbf{v}}^\top \underline{\mathbf{x}}^{(t+1)} - \underline{\mathbf{v}}^\top \underline{\mathbf{x}}^{(t)} \qquad // \text{ TD-error } \Delta v \text{ at time } t$$

$$\underline{\mathbf{v}} \leftarrow \underline{\mathbf{v}} + \eta \, \Delta v \, \underline{\mathbf{e}} \qquad // \text{ update all visited states}$$

$$\underline{\mathbf{e}} \leftarrow \gamma \, \lambda \, \underline{\mathbf{e}} + \underline{\mathbf{x}}^{(t+1)} \qquad // \text{ update eligibility trace } \underline{\mathbf{e}}$$

end



- TD(0): TD learning as defined before

RMS averaged over 100 random-walks on a 19-state chain, rewarded at one end

(Sutton and Barto, 1998)

Contraction properties of TD learning

- asynchronous TD update at time t for all states $\underline{\mathbf{x}}_i$:
  - let $v_t := \underline{\mathbf{v}}^\top \underline{\mathbf{x}}^{(t)}$ and $\mu_{it} = \underline{\mathbf{x}}_i^\top \underline{\mathbf{x}}^{(t)}$

$$\hat{B}_t^\pi[\underline{\mathbf{v}}]_i \;:=\; v_i + \eta \underbrace{\mu_{it}\left(r^{(t)} + \gamma v_{t+1} - v_t\right)}_{\text{TD-error } \Delta v_t \text{ if } \underline{\mathbf{x}}_i = \underline{\mathbf{x}}^{(t)}}$$

- $\hat{B}_t^\pi$ is *in general* a **non-expansion**

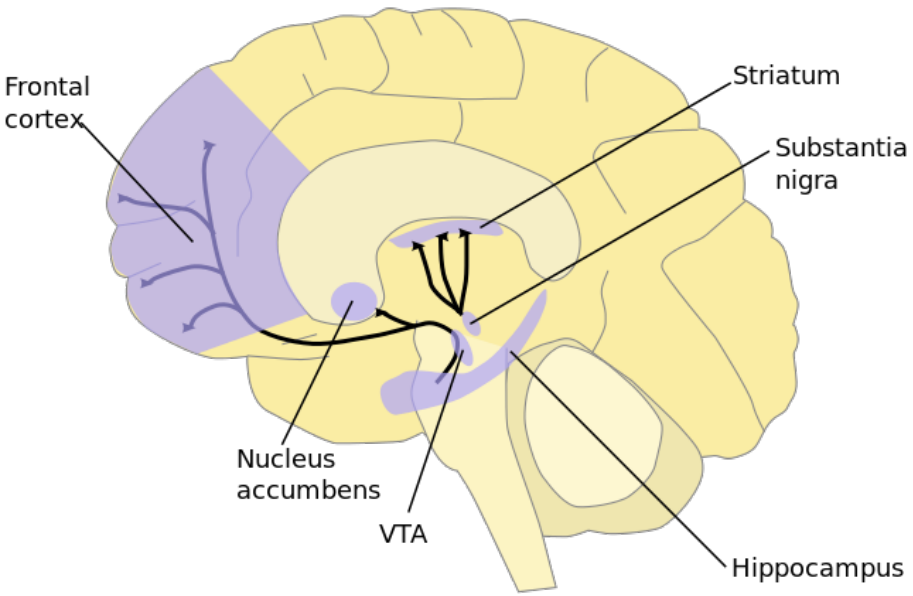
$$\max_{1\leq i\leq S} \left|\hat{B}_t^\pi[\underline{\mathbf{v}}]_i - \hat{B}_t^\pi[\underline{\mathbf{w}}]_i\right| \quad\leq\quad \max_{1\leq i\leq S} |v_i - w_i|$$

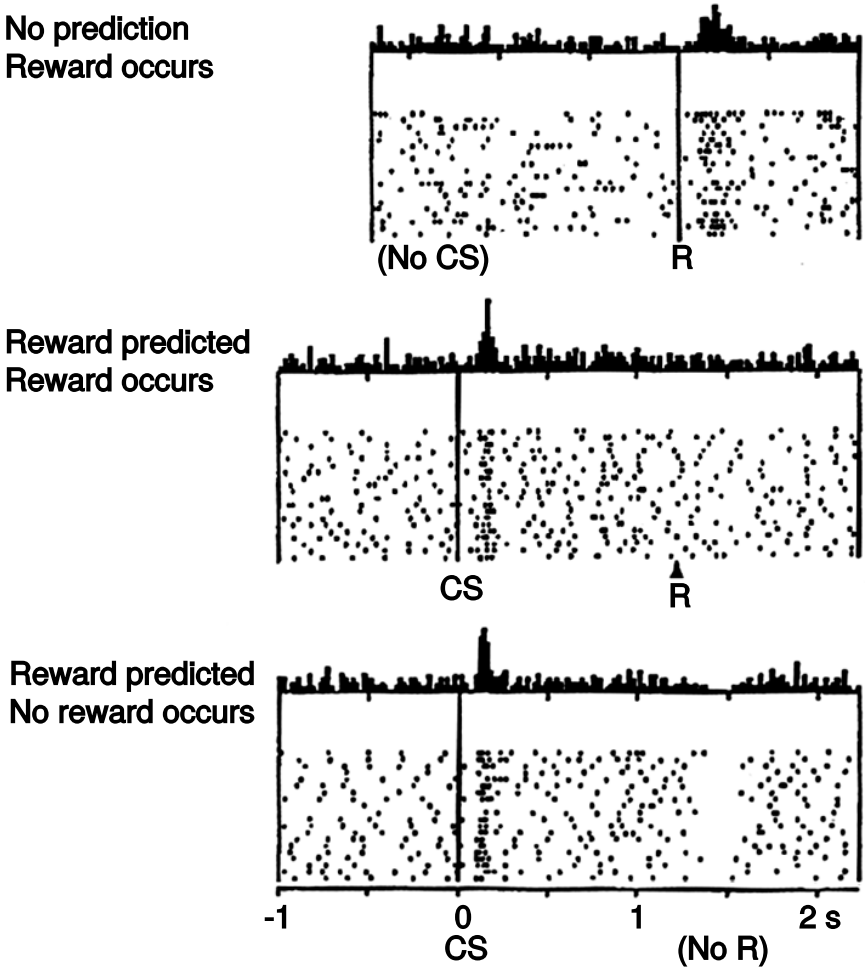
- $\hat{B}_t^\pi$ is *sometimes* a **contraction mapping**
  - in states $\underline{\mathbf{x}}^{(t)}$ with $|v_t - w_t| \geq \max_{i\neq t} |v_i - w_i|$

$$\left|\hat{B}_t^\pi[\underline{\mathbf{v}}]_t - \hat{B}_t^\pi[\underline{\mathbf{w}}]_t\right| \quad\leq\quad (1-\eta(1-\gamma))\,|v_t - w_t|$$

Neurological relevance of reinforcement learning

- dopamine neurons encode TD-errors in many species

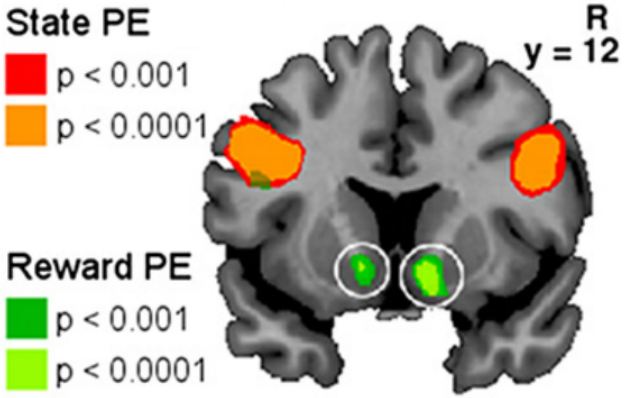




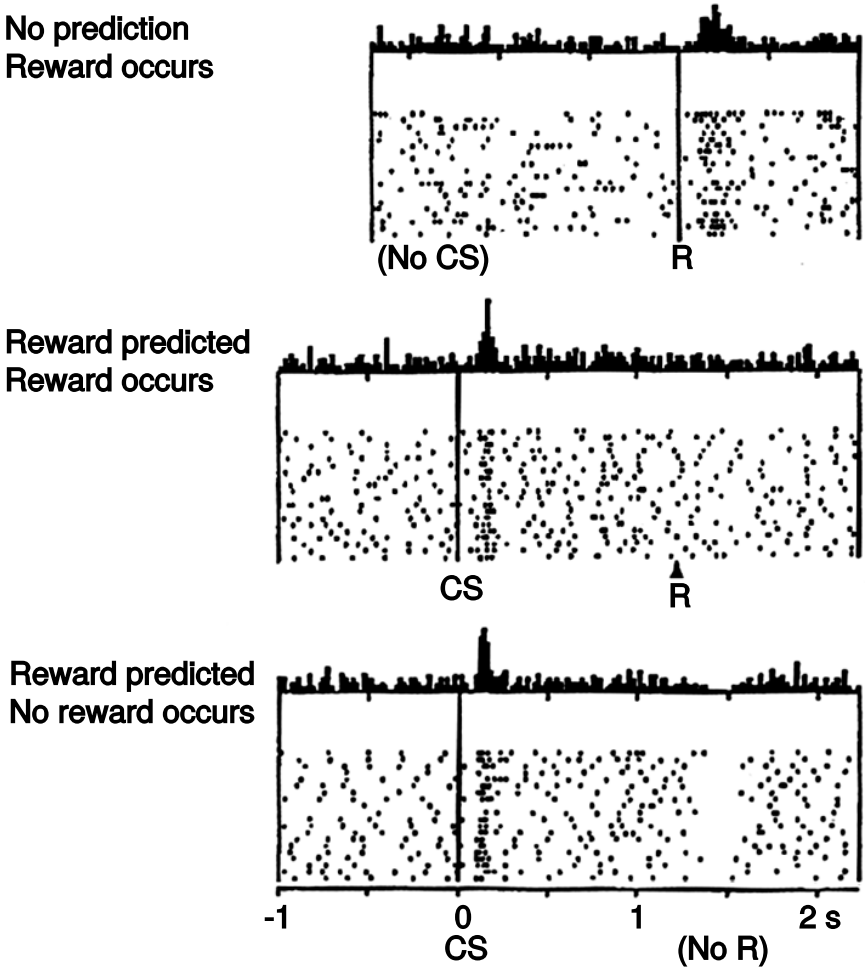
(Schultz et al., 1997)

Neurological relevance of reinforcement learning

- dopamine neurons encode TD-errors in many species
- model-based prediction errors were found in human *pre-frontal cortex*

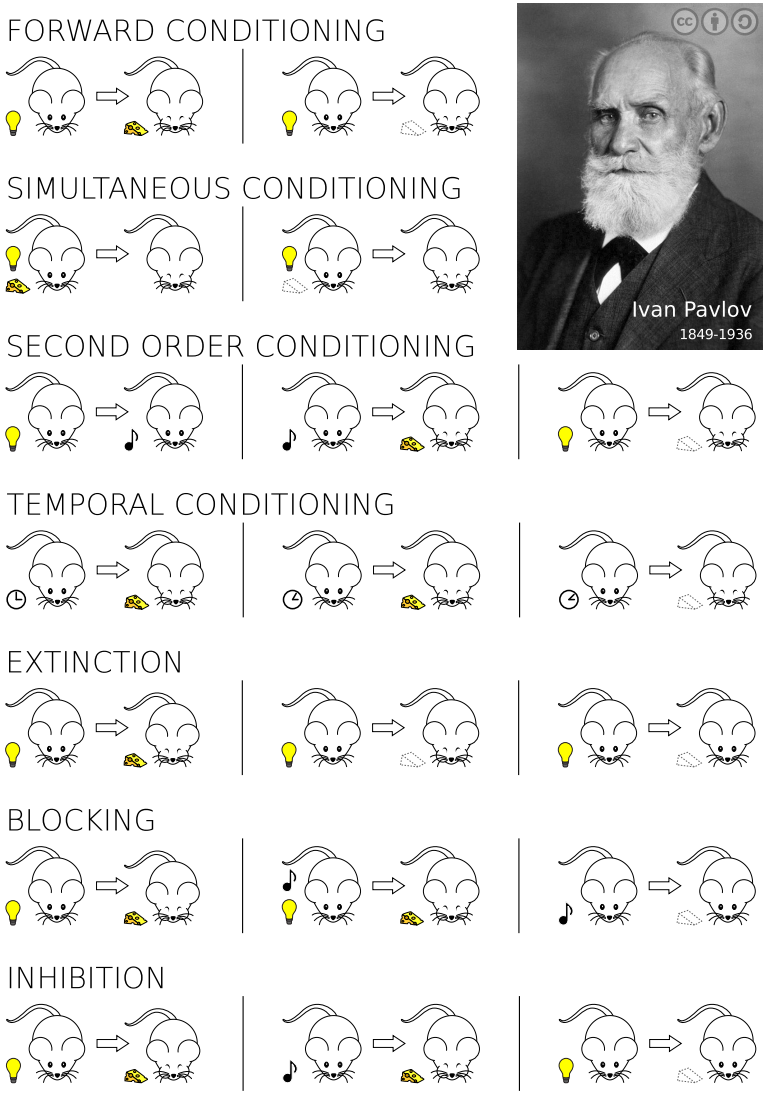


(Gläscher et al., 2010)



(Schultz et al., 1997)

The many faces of classical conditioning



References I

D. P. Bertsekas. *Dynamic Programming and Optimal Control*, volume 2. Athena Scientific, 3rd edition, 2007.

Peter Dayan. The role of value systems in decision making. In *Better Than Conscious?: Decision Making, the Human Mind, and Implications for Institutions*. MIT Press, 2008. ISBN 0-262-19580-1.

Peter Dayan and Yael Niv. Reinforcement learning: The good, the bad and the ugly. *Current Opinion in Neurobiology*, 18:185–196, 2008.

Jan Gläscher, Nathaniel Daw, Peter Dayan, and John P. O'Doherty. States versus rewards: Dissociable neural prediction error signals underlying model-based and model-free reinforcement learning. *Neuron*, 66:585–595, 2010.

Wolfram Schultz, Peter Dayan, and P. Read Montague. A Neural Substrate of Prediction and Reward. *Science*, 275(5306):1593–1599, March 1997. ISSN 1095-9203.

Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, 1998.